Guia de estudos *Data Science*



Neuron/USP

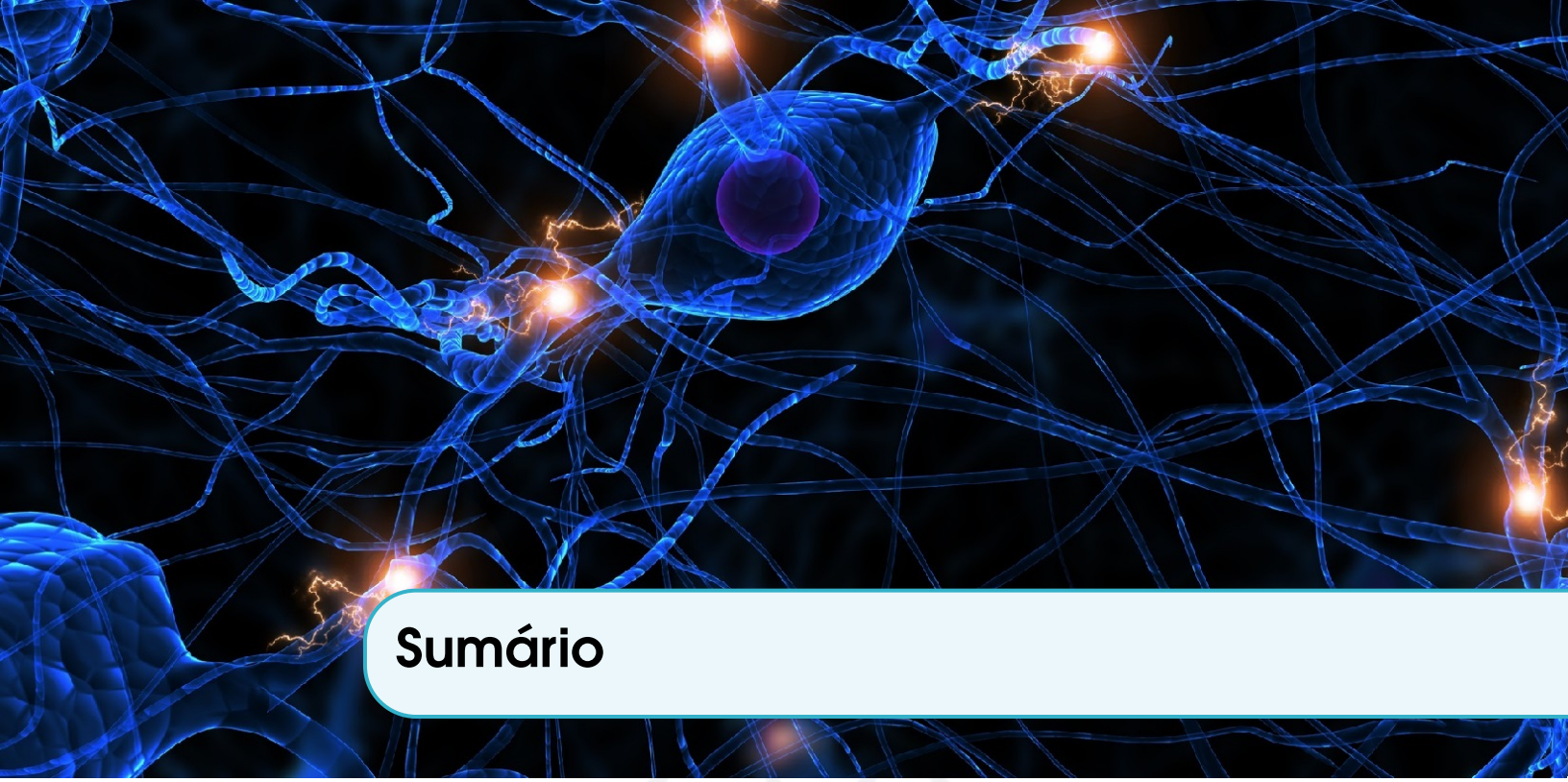
GRUPO DE ESTUDOS EM *Data Science* NEURON/USP

FACEBOOK.COM/NEURONUSP

Este material, em conjunto com os Notebooks do Jupyter que o acompanham, é de propriedade do grupo Neuron/USP e, legalmente, não pode ser reproduzido para fins comerciais (sujeito a processos judiciais). Qualquer dúvida ou sugestão entre em contato conosco pela nossa página do Facebook ou e-mail: *neuron.conteudo@gmail.com*.

Autores: Leonardo Ernesto, Samuel Henrique e Felipe Maia Polo.

Edição: 16 de abril de 2018



Sumário

1 Introdução 5

1.1 O grupo 5

1.2 O conteúdo 5

1.3 Os autores 6

1.4 Gestão 2018 7

2 Programação em Python 9

2.1 Instalação Python 3.6 9

2.1.1 Instalação no Windows 9

2.1.2 Instalação no Linux 10

2.1.3 Instalação no Mac 10

2.2 Instalando PIP 10

2.3 Jupyter Notebook 11

2.3.1 Instalação no Windows 11

2.3.2 Instalação no Linux/Mac 11

2.3.3 Conhecendo o Jupyter Notebook 12

2.4 Materiais - Introdução ao Python 13

2.5 Exercícios complementares ao conteúdo de Introdução ao Python 13

2.6 Exercícios de Desenvolvimento em Python 14

2.6.1 Iniciante 14

2.6.2 Intermediário 15

2.6.3 Avançado 15

3 Fundamentos de Matemática 17

3.1 Estatística 17
3.1.1 O que é Estatística? 17
3.1.2 Variáveis aleatórias 18
3.1.3 Distribuição de uma variável aleatória 18
3.1.4 Estatística: da teoria à prática 19
3.1.5 Média 20
3.1.6 Mediana 20
3.1.7 Variância 21
3.1.8 Desvio-Padrão 21
3.1.9 Covariância 22
3.1.10 Coeficiente de Variação 22
3.1.11 Coeficiente de Correlação de Pearson 22

3.2 Álgebra Linear 23
3.2.1 Matrizes 23
3.2.2 Vetores 23
3.2.3 Operações com matrizes e vetores 24
3.2.4 Matrizes especiais 25
3.2.5 Produto escalar e norma euclidiana 25
3.2.6 Distância entre vetores 25
3.2.7 *Machine Learning - K Nearest Neighbours* para classificação 26
3.2.8 Materiais complementares 29

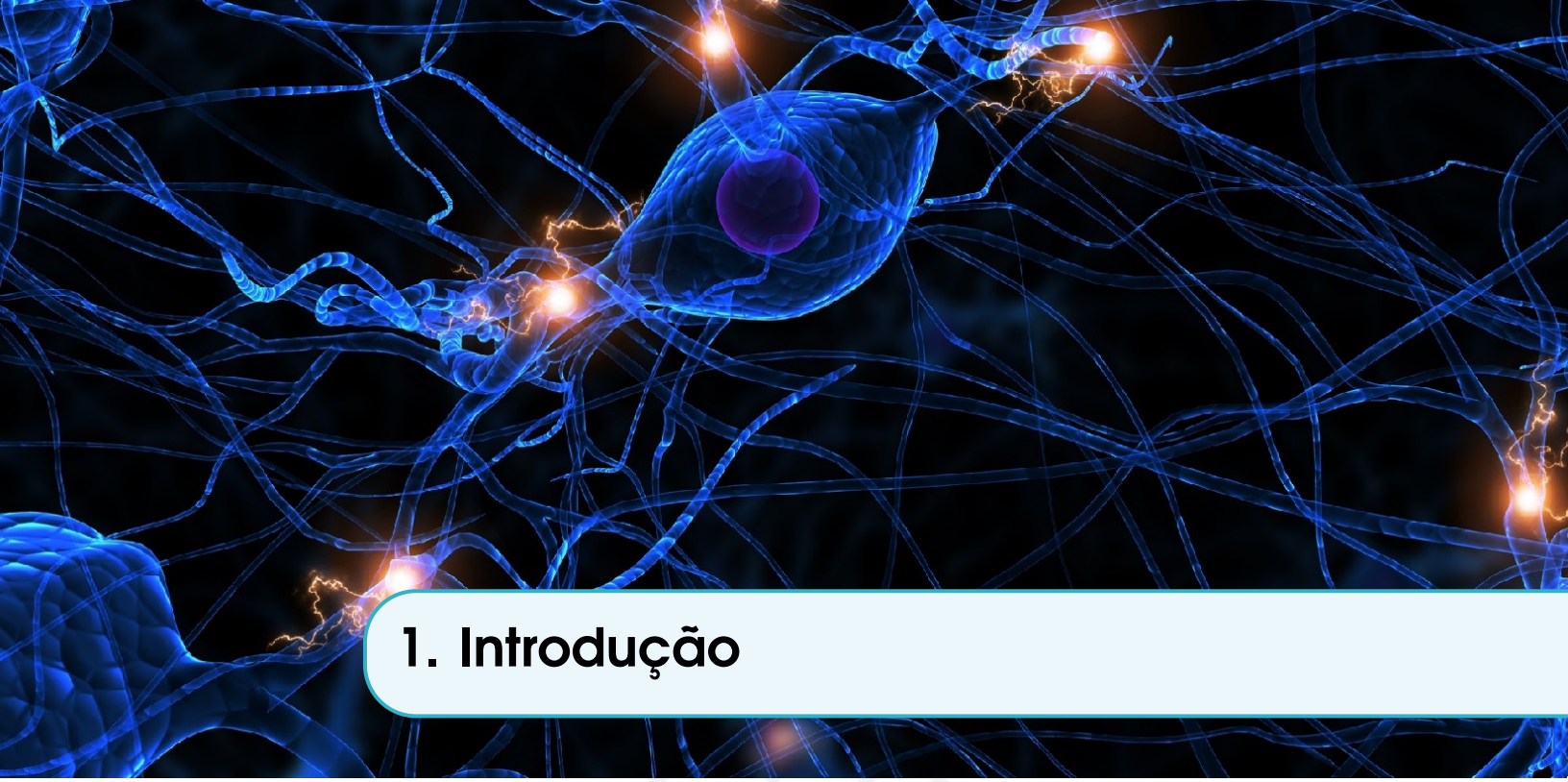
3.3 Otimização de funções 29
3.3.1 O conceito de Derivadas 29
3.3.2 Otimizando uma função na prática 31
3.3.3 Otimização de funções com mais de uma variável real 33
3.3.4 *Machine Learning* - Ajuste de curvas entre pontos 34
3.3.5 Observações finais 36
3.3.6 Materiais complementares 36

4 Aprendizado de Máquina 37

4.1 Tipos de Aprendizado de Máquina 37

4.2 O Aprendizado Não Supervisionado 37
4.2.1 Métodos de Aprendizado Não Supervisionado 37
4.2.2 Métodos de Particionamento 38
4.2.3 Classificação Hierárquica 40
4.2.4 *Principal Components Analysis* - PCA 41

4.3 Materiais complementares 42



1. Introdução

1.1 O grupo

O grupo Neuron surge com o principal intuito de cobrir um *gap* existente na realidade dos brasileiros e até mesmo das principais universidades brasileiras: o estudo, discussão e desenvolvimentos de projetos relacionados ao *Data Science*, que engloba programação, análise de dados e o emprego das tecnologias de Aprendizado de Máquina com fins práticos ligados tanto à pesquisa quanto aos negócios. Como atividade principal, o grupo Neuron abrigará um grupo de estudos em *Data Science* a partir do ano de 2018 localizado no campus da Universidade de São Paulo (USP) em Ribeirão Preto, São Paulo. No que tange às atividades extras, traremos periodicamente profissionais e estudiosos da área para conversar com nossos membros e daremos apoio a alunos que queiram desenvolver projetos próprios. Além disso, fechamos parceria com o grupo HAIT, mais importante grupo de estudos em Inteligência Artificial formado por alunos das melhores universidades japonesas. Essa parceria possibilitará grande troca de experiência e *networking* entre os membros dos dois grupos, sendo que o desenvolvimento de projetos entre alunos de diferentes nacionalidades será o ponto alto. Espero que aproveite seu tempo no Neuron. Grande abraço,

Felipe Maia Polo - Presidente 2018

1.2 O conteúdo

O conteúdo do Neuron foi desenvolvido com a finalidade de maximizar o seu aprendizado em *Data Science*, utilizando a linguagem de programação Python com o enfoque em Aprendizado de Máquina. Os assuntos que serão abordados foram divididos em quatro grandes tópicos: Introdução a Programação em Python, Fundamentos de Matemática, *Machine Learning* e *Deep Learning*. Esses tópicos serão divididos em subtópicos, para melhor explicação dos mesmos. Os conteúdos serão abordados em aulas expositivas com exercícios práticos, que serão disponibilizados antes de cada aula. Também contaremos com palestras de profissionais das diversas áreas, com o intuito de propiciar aos membros do grupo um conhecimento amplo sobre as aplicações das tecnologia no estudos em *Data Science*. Todo o conteúdo foi feito com muito esforço e dedicação com a

colaboração do Felipe Maia Polo e do Samuel Henrique. Espero que o conteúdo seja completo e de fácil compreensão. Atenciosamente,

Leonardo Ernesto – Diretor de Conteúdo 2018

1.3 Os autores

Este espaço é dedicado para que você conheça um pouco dos nossos autores. No presente momento, todos são alunos da Universidade de São Paulo em Ribeirão Preto.

Felipe Maia Polo

Estudante de Economia pela Faculdade de Economia, Administração e Contabilidade de Ribeirão Preto da Universidade de São Paulo. Tem experiência com Estatística, Econometria e Aprendizado de Máquina. Trabalhou no Laboratório de Estudos e Pesquisas em Economia Social (LEPES) focando majoritariamente em avaliação de políticas públicas na área da educação. Concluiu parte da graduação na Universidade de Tóquio, onde teve contato com grupos voltados ao ensino e treinamento de pessoas em *Data Science*, estes que foram grandes inspirações para a concretização do Grupo de Estudos em *Data Science* Neuron/USP.

Leonardo Ernesto

Estudante de Informática Biomédica pela Faculdade de Medicina de Ribeirão Preto e pela Faculdade de Filosofia Ciências e Letras de Ribeirão Preto na Universidade de São Paulo. Exerce o cargo de diretor de assuntos acadêmicos no Centro Estudantil da Informática Biomédica - CEIB, o qual está sob sua responsabilidade desde 2016 e já foi representante Discente na Comissão Coordenadora da IBM em 2016. É aluno de iniciação cientifica desde 2014 no Laboratório de Tráfego Intracelular de Proteínas, onde desenvolve pesquisa de interação de proteínas. Possui amplo conhecimento nas áreas de Bioinformática, Inteligência Artificial, Reconhecimento de Padrões e *Data Mining*.

Samuel Henrique

Estudante de Informática Biomédica pela Faculdade de Medicina de Ribeirão Preto e pela Faculdade de Filosofia Ciências e Letras de Ribeirão Preto na Universidade de São Paulo. Conhecimento avançado na linguagem Python 3.6, algumas de suas bibliotecas e Programação Orientada a Objetos.

1.4 Gestão 2018

Presidente
Felipe Maia Polo

Vice-Presidente
Jackeline Garcia

Conteúdo
Diretor: Leonardo Ernesto *Membro:* Samuel Henrique

Desenvolvimento de pessoas
Diretor: Igor Soares Costa *Membro:* João Lavinas

Financeiro
Diretor: Natasha Freitas *Membro:* Lucas Negreiros

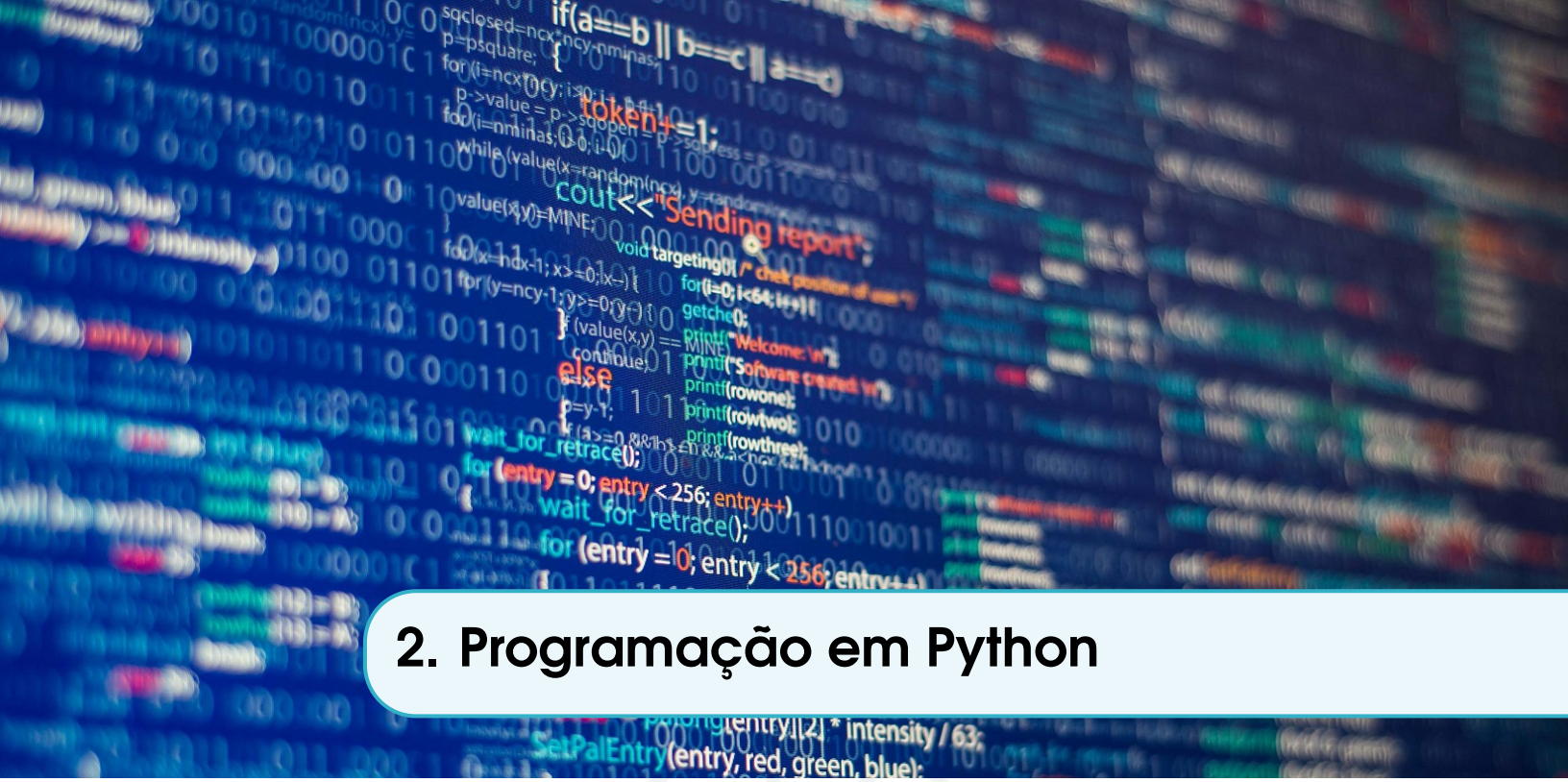
Marketing
Diretor: Gustavo Zequini *Membro:* Jonathan Batista

Projetos
Diretor: Gustavo Ribeiro

Relações Públicas
Diretor: Henrique Silva *Membro:* Lucas Negreiros

neuron

neuron



2. Programação em Python

2.1 Instalação Python 3.6

2.1.1 Instalação no Windows

- Para instalar o Python no sistema operacional Windows, acesse o site `www.python.org`. Depois clique em *Download* e selecione a versão 3.6.4.
- Ao terminar o download clique em executar, coloque tudo em *default*, que seria as configurações padrões já inseridas pelo instalador. Clique em finalizar e pronto: o interpretador do Python estará instalado no computador.

Agora vamos conferir se o python foi instalado com sucesso:

- Clique na tecla "Windows"e "r"simultaneamente ou clique com o botão direito em cima do menu iniciar e depois clique em executar;
- No Executar digite "cmd"e clique no Enter;
- Com isso abrimos o *prompt* de comando do Windows. Para verificar se o Python foi realmente instalado basta digitar "python"e dar Enter. Logo após, a seguinte mensagem deve aparecer no *prompt*:

Caso não consiga executar o Python no prompt de comando, o *path* (caminho da pasta na qual estão os arquivos de Python) não deve estar configurado. Basta seguir os passos a seguir:

- Vá em Computador, clique com o botão direito, e depois clique em Propriedades;
- Agora, clique em Configurações Avançadas de Sistema e, a seguir, em Variáveis de Ambiente;
- Desça até encontrar a variável Path, clique em Editar;
- Na janela aberta, no campo Valor de Variável, vá ao final da lista que aparece, acrescente um ponto e vírgula “;” e adicione o endereço/caminho de onde a pasta do Python está instalada em seu computador.

Abaixo estão alguns links caso ocorra problemas ou dúvidas no processo de instalação:

- `https://www.youtube.com/watch?v=gDxt7mncWys`
- `https://tutorial.djangogirls.org/pt/python_installation/`
- `http://excript.com/python/como-instalar-o-python-windows.html`

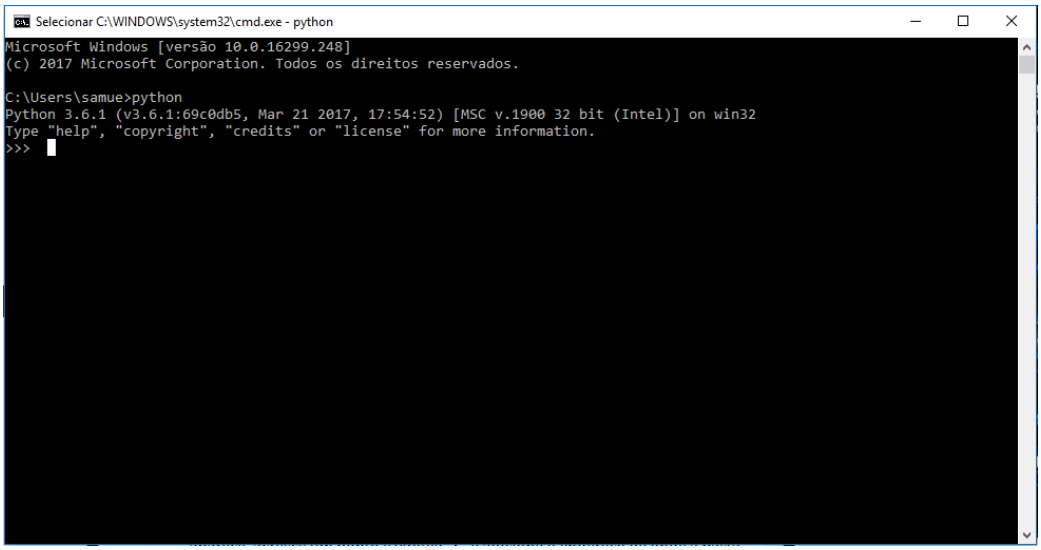


Figura 2.1: Verificando o sucesso da instalação do Python

2.1.2 Instalação no Linux

- Abra o terminal do Linux.
- Digite o comando **sudo apt-get install python3**.
- Digite python3 no terminal para conferir se a instalação foi realizada corretamente.

2.1.3 Instalação no Mac

No Mac é comum o Python já vir instalado de fábrica. Para conferir isso, basta digitar **witch python3** no terminal do Mac, onde deve aparecer uma mensagem parecida com essa "/usr/bin/python". Caso isto não aconteça, basta digitar **brew install python3** no terminal.

2.2 Instalando PIP

O PIP é uma biblioteca do Python, que tem como finalidade nos ajudar a instalar outras bibliotecas com maior facilidade e rapidez. Sua instalação[1] pode ser realizada de forma simples e é feita da mesma maneira em qualquer sistema operacional, como veremos a seguir:

- Primeiro acesse o site `https://pip.pypa.io/en/stable/installing/#id8`;
- Clique com o botão direito no **get-pip.py.[2]**;
- Depois em clique em "Salvar link como...", e selecione para salvar na sua pasta de Downloads:
- Agora vamos abrir novamente o prompt de comando e digitar **cd Downloads**;
- Por fim, na pasta Downloads basta digitar **python get-pip.py** no prompt de comando.

Links adicionais caso ocorra problemas na instalação:

- `https://www.makeuseof.com/tag/install-pip-for-python/`.
- `https://stackoverflow.com/questions/4750806/how-do-i-install-pip-on-windows`.

[1] Esta parte é focada no Windows. Se você estiver em um Linux ou Mac, por favor abrir primeiro o link disponibilizado mais a frente.

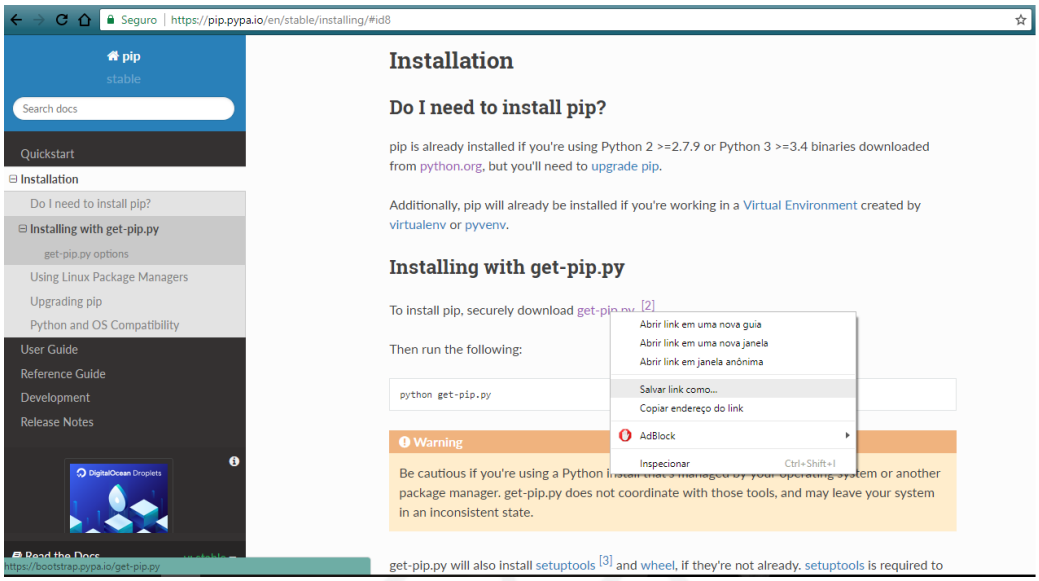


Figura 2.2: Site PIP

2.3 Jupyter Notebook

A plataforma de programação que usaremos nesse curso é a Jupyter Notebook (`http://jupyter.org/index.html`). Essa plataforma será o local onde iremos escrever e rodar nosso código em Python. O Jupyter Notebook apresenta uma interface intuitiva, na qual podemos rodar nossos códigos por blocos de comandos, facilitando o uso. Dessa maneira, é possível ver se determiando pedaço do código realiza a ação que desejamos. Como informação adicional, é legal dizer que o Jupyter roda diretamente no seu navegador com servidor local.

2.3.1 Instalação no Windows

Para instalar o Jupyter Notebook, basta acessar o link: `https://www.anaconda.com/download/` e clicar em "Download". Em seguida, ir seguindo os passos de instalação sem alterar a configuração padrão. Após o Anaconda estar instalado em seu computador, basta ir até a pasta onde o programa se encontra e clicar no **Anaconda Prompt**. Em seguida, no prompt digitar **jupyter notebook** e dar Enter.

2.3.2 Instalação no Linux/Mac

Para instalar o Jupyter Notebook, basta acessar o link: `https://www.anaconda.com/download/` e clicar em "Download". Em seguida no prompt de comando, digite os seguintes comandos: **python3 -m pip install –upgrade** e em seguida **pip python3 -m pip install jupyter**. Para abrir o programa basta digitar **juypter notebook** no prompt de comando. Com isso teremos o Jupyter instalado, caso aparacer algum erro olhe alguns dos links abaixo para ajudá-lo:

- `http://jupyter.readthedocs.io/en/latest/install.html`
- `http://jupyter.org/install`
- `https://www.youtube.com/watch?v=qRixkfuYp88`



Figura 2.3: Prompt na pasta Downloads

2.3.3 Conhecendo o Jupyter Notebook

Na plataforma iremos encontrar todas as pastas do nosso computador. Primeiramente, é importante criar uma pasta onde você trabalhará. Após criar essa pasta no seu computador, ache o caminho dela no Jupyter Notebook e depois clique em **new** no canto superior e em seguida Python 3. Assim será criado seu notebook com Python.

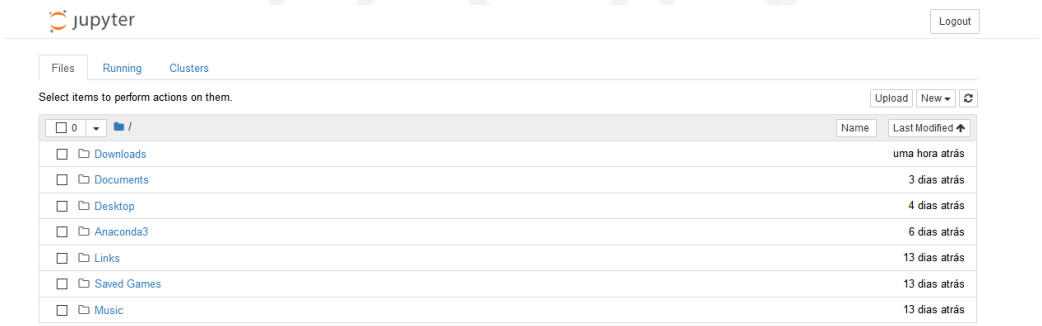


Figura 2.4: Layout da tela inicial do Jupyter Notebook

Com o processo de instalação finalizada, vamos entender melhor o que é um notebook: ele é a plataforma onde faremos nossa programação em Python. Nele que poderemos separar nosso código em células de comando e executá-las separadamente. Conforme você for programando no Jupyter Notebook perceberá que é possível mesclar código (Python) com textos (Markdown/LaTex), que podem deixar os notebooks criados mais autoexplicativos.

Para criar seu primeiro código no Jupter Notebook, basta ir no notebook que foi criado anteriormente, digitar o código a ser executado, podendo ser apenas uma linha de comando quanto várias linhas, em seguida basta clicar em **Run** no canto superior para ter seu código compilado, e conferir o resultado. Caso ocorra erros no compilamento do código, aparecerá uma descrição dos erros.

Para alterar um bloco do Jupyter para aceitar textos, basta alterar a célula na caixa superior onde



Figura 2.5: Primeiro programa no Jupyter Notebook

está escrito **Code** para **Markdown**

Podemos também baixar nossos códigos ou textos, clicando em **File** e depois clicar em **Downloads as**, onde irá aparecer todos os formatos de arquivo no qual podemos baixar o notebook.

Para abrir um notebook externo, como por exemplo, o notebook da aula que será dada, basta navegar entre as pastas até o local onde está salvo o arquivo no seu computador, e então dar dois cliques no arquivo, em seguida o arquivo será aberto.

Links adicionais para aprender a utilizar melhor o Jupyter Notebook:

- `https://www.datacamp.com/community/tutorials/tutorial-jupyter-notebook`.
- `https://www.youtube.com/watch?v=xuahp9g3Dic`.

2.4 Materiais - Introdução ao Python

Os conteúdos disponibilizados na seguinte pasta serão utilizados no nosso primeiro encontro presencial. Na pasta você poderá encontrar um Jupyter Notebook e um pdf que contém o mesmo conteúdo. Basta baixar o notebook e abrí-lo no Jupyter em seu computador:

- **Introdução ao Python I**: `https://drive.google.com/open?id=1Khy_K-1QaKMqxVUeW25xNKc56syh7WiQ`
- **Introdução ao Python II**: `https://drive.google.com/open?id=1fznVGKjGHSyykC5bI5Olv3yxmjdt4xlh`

2.5 Exercícios complementares ao conteúdo de Introdução ao Python

1. Com a frase "Estou aprendendo Python com o Neuron"faça o seguinte: separe palavra por palavra armazenando-as em variáveis distintas e depois concatene todas printando a frase para o usuário.
2. Faça um código onde o usuário entre com a idade e diga se ele pode ou nao tirar licensa de motorista.
3. Crie uma função que diga se um número é ímpar ou par.
4. Crie uma lista apenas com números ímpares e outra somente com números pares e uma com números primos (range (0,50)). Use condicionais e faça o computador trabalhar por você.
5. Crie uma função que receba um número de 1 a 10 e retorne sua tabuada.
6. Printar uma matriz número por número.
7. Faça um jogo da forca, de forma que 2 pessoas joguem e uma delas entra com a palavra. Após isso a outra deverá adivinhar as letras, lembrando que com 6 erros a pessoa perde. O jogo deverá mostrar as letras que já foram tentadas, e conforme os acertos forem acontecendo, o programa deverá mostrar a letra no seu devido espaço até a palavra se formar.
8. Estude a função *random* do Python e após isso crie uma lista de 10 valores preenchidos de forma aleatório. Então:
   - Dê o maior valor da lista;
   - Dê o menor valor da lista;
   - Dê a soma dos valores da lista;

- Dê o tamanho da lista;
- Dê a lista em ordem crescente;
- Dê a lista em ordem decrescente;

9. Peça para o usuário inserir uma frase e faça o seguinte com ela:
   - Tornar todas as letras maiúsculas;
   - Retire os espaços entre as palavras;
   - Realize as duas tarefas conjuntamente.
10. Pegue uma poesia qualquer, coloque em um arquivo .txt e dê print usando a função *readlines*.
11. Com o poema do exercício anterior, escolha uma palavra que aparece com grande frequência e faça um programa que conte quantas vezes essa palavra aparece no poema inteiro;
12. Em seguida, como continuação do exercício anterior, faça um programa que troque a palavra escolhida anteriormente por "Neuron" e depois reescreva o arquivo .txt com as modificações feitas.

2.6 Exercícios de Desenvolvimento em Python

Estes exercícios tem a finalidade de avaliar seu conhecimento e habilidades de programação em Python, resolva quais conseguir.

2.6.1 Iniciante

1. Escreva um algoritmo que armazene o valor 999 na variável *a* e o valor 555 na variável *b*.A seguir (utilizando apenas atribuições entre variáveis) troque os seus conteúdos fazendo com que o valor que está em *a* passe para *b* e vice-versa.
2. Escreva um algoritmo para ler um valor (do teclado) e escrever (na tela) o seu antecessor.
3. Ler um valor e escrever a mensagem **É MAIOR QUE 10!** se o valor lido for maior que 10, caso contrário escrever **NÃO É MAIOR QUE 10!**
4. Ler dois valores e imprimir uma das três mensagens a seguir: 'Números iguais', caso os números sejam iguais 'Primeiro é maior', caso o primeiro seja maior que o segundo; 'Segundo maior', caso o segundo seja maior que o primeiro.
5. Ler três valores (considere que não serão informados valores iguais) e escrever o maior deles.
6. Ler três valores (considere que não serão informados valores iguais) e escrever a soma dos dois maiores.
7. Ler três valores (A, B e C) representando as medidas dos lados de um triângulo e escrever se formam ou não um triângulo. **OBS:** para formar um triângulo, o valor de cada lado deve ser menor que a soma dos outros dois lados.
8. Faça um algoritmo que leia a idade de uma pessoa expressa em anos, meses e dias e escreva a idade dessa pessoa expressa apenas em dias. Considerar ano com 365 dias e mês com 30 dias.
9. Faça um algoritmo para dizer se um número é par ou impar.
10. Faça um jogo onde é sorteado um número aleatório de 0 a 100(incluindo 100) e o jogador deve tentar acertar este número, a cada tentativa do jogador devera ser informado o seguinte:
    - Parabéns você ganhou!(caso ele acerte o número)
    - O número é maior! (caso o número for maior)
    - O número é menor! (caso o número for menor)
    - Faça um menu de entrada para o jogo

    Opcional: Faça que ao final do jogo seja possivel jogar denovo sem ter que fechar o prompt**Dica:** Estudar função Random e a sua seed.

2.6.2 Intermediário

1. Faça um algoritmo que preencha uma matriz e print número por número, depois faça uma matriz binária (apenas 0 e 1) e faça que print apenas os número 1 e onde seja 0 esteja vazio.
2. Faça uma função que print o Fibonacci de um número. **Opcional:** fazer sem recursão.
3. Concatenar duas listas em ordem crescente. Faça para listas de tamanhos iguais e diferentes
4. Faça um algoritmo que multiplique duas matrizes se possível (faça para matrizes não quadráticas).
5. Faça uma função que:
   - Leia arquivos txt
   - Edite arquivos txt
   - Salve arquivos txt
6. Dado uma lista de tamanho X , preenchidas de forma não ordenada, faça um algoritmo que faça uma busca por um valor , caso ele nao esteja na lista mande uma mensagem falando isso. **Dica:** Primeiro ordene a lista.
7. Faça um função que descobre se um número é primo.
8. A prefeitura de uma cidade deseja fazer uma pesquisa entre seus habitantes. Faça um algoritmos para coletar dados sobre o salário e número de filhos de cada habitante e após as leituras, escrever:
   - Média de salário da população
   - Média do número de filhos
   - Maior salário dos habitantes
   - Percentual de pessoas com salário menor que R$ 150,00

   Faça uma função para as letras A e B, outra para C e uma para D, faça um menu para escolher qual deve ser usada, e deixe uma opção para o cadastro dos habitantes e faça que seja salvo em um arquivo.

2.6.3 Avançado

Usando conceito de Programação Orientada a Objetos resolva os exercícios a seguir:

1. Faça um algoritmo que resolva o jogo resta um. **Dica:** pesquise sobre busca em profundidade.
2. Dado uma matriz labirinto como na imagem abaixo, faça um algoritmo que diz se um labirinto tem ou nao saída. **Opcional:** Fazer uma função que mostra o caminho de saída do labirinto.

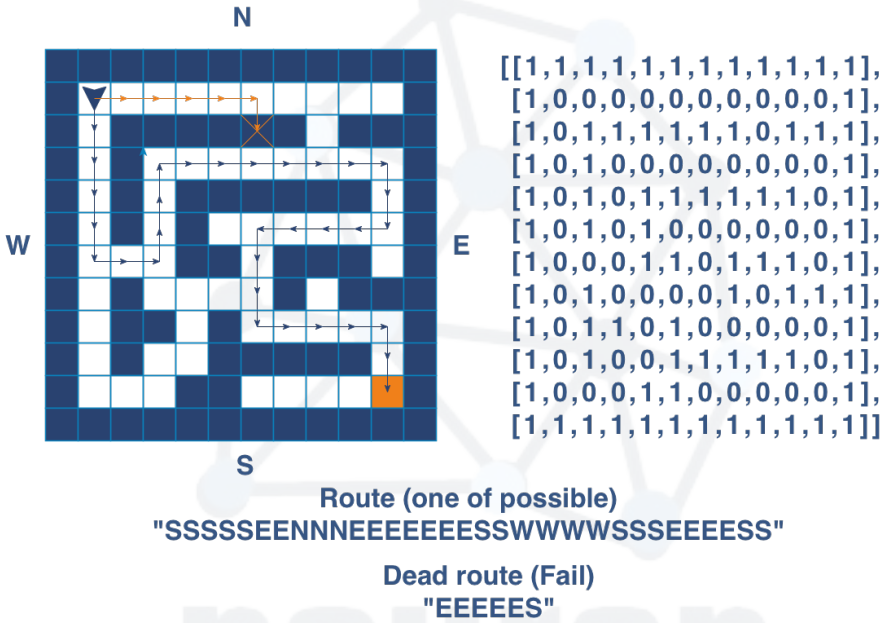
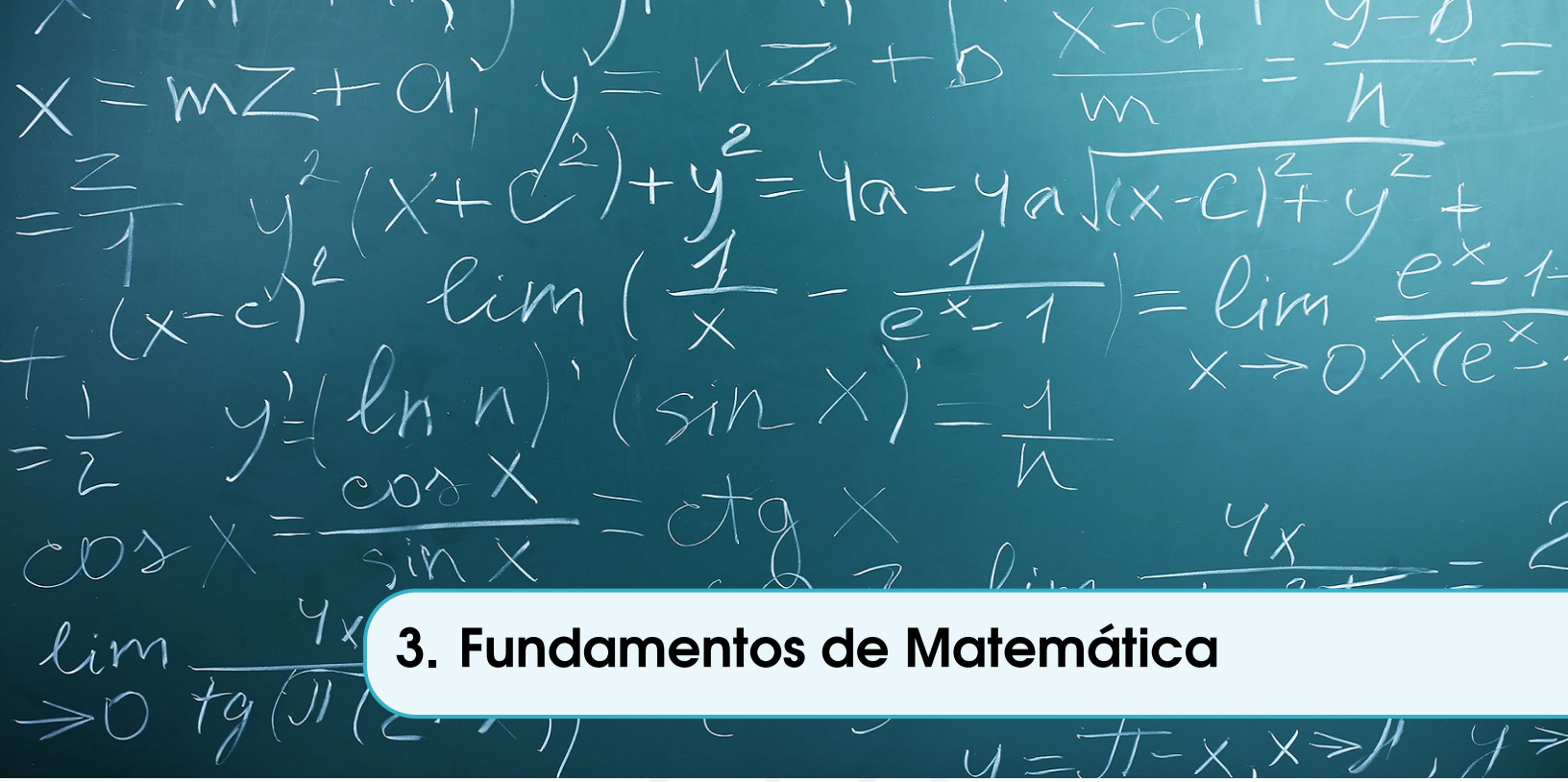


Figura 2.6: Labirinto



3. Fundamentos de Matemática

Por que é importante voltar aos contextos matemáticos para entender *Machine Learning* e outras tecnologias de nosso interesse? A resposta é simples e direta: os algoritmos *Machine Learning* são construídos em cima desses conceitos. Os campos mais importantes e que vamos ver rapidamente são os da Estatística, Álgebra Linear e Cálculo. É importante ressaltar que na apostila você encontrará uma visão superficial sobre as técnicas matemáticas necessárias para avançarmos, no entanto, se você decidir se aprofundar seus estudos, recomendamos fortemente que também estude bastante matemática, pois esta será necessária.

3.1 Estatística

Nesta parte do conteúdo veremos conceitos de estatística, a importância e suas aplicações na análise de dados, para auxiliar na sua interpretação. Trataremos de medidas de tendência centrais e de dispersão, e formulação de gráficos no Python. Será apresentado também o conceito de Teste de Hipóteses juntamento com as técnicas possiveis para sua validação. O estudo desses tópicos se faz de grande importância para a compreenção dos dados e formulação de objetivos coesos e representativos.

3.1.1 O que é Estatística?

Estatística é um conjunto de ferramentas, que nos ajudam a entender melhor características de uma certa população, seja de pessoas ou não. Por exemplo, queremos saber qual a altura média das pessoas em uma sala de aula ou até mesmo no Brasil inteiro. No primeiro caso, é fácil e pouco custoso perguntar a todas as pessoas da sala de aula quais são suas alturas individuais e tirar uma média no final. Por outro lado, no segundo caso, podemos considerar que é praticamente impossível saber as alturas individuais de todos os brasileiros para depois tirar uma média dos números. Quando não temos acesso a dados de toda uma população, podemos os utilizar das técnicas de *inferência estatística*, que são técnicas voltadas a inferir características de uma certa população somente tendo acesso a uma **amostra** dela, que no fundo é um subconjunto da população inteira. Uma coisa importante aqui é frisar que, na maioria dos casos, assumimos que a amostra que temos acesso é

aleatória, ou seja, todos os indivíduos da população tem igual probabilidade de ser escolhido para compor o sub-conjunto[1].

3.1.2 Variáveis aleatórias

A probabilidade é uma medida relativa a um Evento específico, que está inserido em um Espaço Amostral[2]. Em outras palavras, a probabilidade, em uma perspectiva clássica, mede o quão provável um evento é dadas certas circunstâncias. Para clarear essa ideia, vamos voltar ao exemplo das alturas: suponha que em nossa sala de aula tenhamos 10 alunos, sendo que 3 deles tenham mais de 1,80m. A probabilidade de se tirar aleatóriamente um aluno que tenha mais de 1,80m é $\frac{3}{10}$. Nesse caso, nosso Evento é tirar um aluno com mais de 1,80m e o Espaço Amostral são os 10 alunos que estão na sala, sendo que apenas 3 têm mais de 1,80m. Se, por exemplo, um desses 3 alunos for substituído por um de 1,70m, o Espaço Amostral muda, alterando também a probabilidade do nosso Evento.

O conceito de Variável Aleatória[3] vem e é fundamental no campo da Probabilidade. Uma Variável Aleatória é uma função que conecta um Evento contido no Espaço Amostral aos números reais. Voltando ao exemplo da sala de aula, considere a variável aleatória X[4] como sendo uma variável binária que assume 1 caso se tire uma pessoa com altura maior do que 1,80m e 0 caso contrário. Então dizemos que a probabilidade de tirarmos uma pessoa maior que 1,80m no caso em que temos 3 pessoas com altura superior a 1,80m entre as 10 pessoas que estão na sala é $P(X = 1) = \frac{3}{10}$. Um outro exemplo de variável aleatória seria Y, que representa o número de pessoas com mais de 1,80m escolhidas sucessivamentes em 2 tentativas. Se consideramos a situação com reposição[5], a probabilidade de se tirar 2 pessoas com mais de 1,80m de altura é igual a $P(Y = 2) = \frac{9}{100}$. Na situação sem reposição, teríamos que $P(Y = 2) = \frac{3}{10}\frac{2}{9} = \frac{1}{15}$.

Em um contexto mais prático, podemos pensar em alguns eventos que podem ser representados por variáveis aleatórias e que são interessantes no campo do *Data Science*: o salário do brasileiro, gênero das pessoas, idade, nível educacional, uma peça de uma máquina estragar ou não, o números de gols em uma partida de futebol etc. Nos exemplos de eventos citados acima, podemos considerar que o contradomínio da função que é uma variável aleatória são os números reais em todos os casos. No entanto, a imagem dessa função varia de acordo com as especificidades de cada caso. Por exemplo, a imagem da função que liga os salários aos números reais são, teoricamente, os reais não-negativos e a imagem da função que liga os gols de uma partida aos reais é, teoricamente, os números naturais.

3.1.3 Distribuição de uma variável aleatória

Como vimos anterioemente, é possível atribuir probabilidades aos eventos contidos em um Espaço Amostral e, assim, também é possível dar probabilidades aos diversos valores, ou intervalos, que uma variável aleatória pode assumir. Antes de proseguirmos é necessário fazer a distinção entre uma variável aleatória discreta[6] e uma variável aleatória contínua[7]: basicamente a diferença entre esses dois tipos de variáveis está em seu contradomínio, sendo que se esse conjunto for um conjunto de

[1] Mais formamelte dizemos que as observações são independentes e identicamente distribuídas. Isso implica que na hora de se amostrar, "sacamos"os indivíduos com reposição.

[2] `https://en.wikipedia.org/wiki/Sample_space`

[3] `https://en.wikipedia.org/wiki/Random_variable`

[4] Geralmente são denotadas por letras maiúsculas.

[5] Ou seja, após escolhermos uma pessoa a retornamos na sala de aula.

[6] `https://en.wikipedia.org/wiki/Probability_mass_function`

[7] `http://www.portalaction.com.br/probabilidades/23-variavel-aleatoria-continua`

"pontos isolados"temos uma variável discreta e se esse conjunto é contínuo, temos uma variável contínua. Exemplos de variáveis discretas são: gênero das pessoas, nível educacional, uma peça de uma máquina estragar ou não, o números de gols em uma partida de futebol. Exemplos de variáveis contínuas são: o salário do brasileiro e a altura das pessoas. É possível observar que variáveis contínuas podem ser "discretizadas", mas o contrário não é verdade.

Quando falamos em variáveis aleatórias discretas, consequentemente falamos de uma *função probabilidade*[8] que é uma função que liga um valor assumido pela variável aleatória a uma probabilidade. Se X é a face de um dado jogado (não viciado), sua função probabilidade pode ser expressa como $P(X = x)$ ou $p(x)$ sendo que X é a variável aleatória e x são possíveis valores da mesma, ou seja, elementos de seu domínio. No caso, o domínio é {1,2,3,4,5,6}:

x	P(X=x)
1	0,167
2	0,167
3	0,167
4	0,167
5	0,167
6	0,167

Por outro lado, quando tratamos de variáveis aleatórias contínuas, trocamos a função probabilidade pela *função densidade*[9]. Ao contrário da função probabilidade, a função densidade não retorna a probabilidade associada a um valor específico, mas sim a densidade associada àquele valor. Na prática, isso que dizer que quando usamos variáveis aleatórias contínuas e o conceito de funções densidade, a probabilidade de uma variável contínua Y de assumir um valor específico é sempre igual a zero e, por isso, trabalhamos com intervalos. Ou seja, ao invés de perguntarmos qual a probabilidade de uma pessoa ter exatamente 1,80m (que é igual a zero), perguntaríamos "qual a probabilidade de uma pessoa ter entra 1,80m e 1,85m de altura?". Se, por exemplo, dizemos que a variável Z tem distribuição Normal Padrão[10], a densidade $f(z)$ é dada por:

$$f(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}} \tag{3.1}$$

Para calcular a probabilidade de Z estar entre -1 e 2, ou seja, $P(-1 \leq Z \leq 2)$ basta calcular a área delimitada pelas retas $z = -1$, $z = 2$, $y = 0$ e pela função densidade $f(z)$[11]. Assim é possível perceber se calcularmos a área entra a função e o eixo z, com z variando de menos a mais infinito, esta vai ser igual a 1. O gráfico da função densidade nesse caso seria:

3.1.4 Estatística: da teoria à prática

Até o presente momento, focamos muito em conceitos da Teoria da Probabilidade, mas não vimos como fazer a transição dessa parte mais teórica à parte prática. É importante ressaltar que raramente teremos acesso à distribuição das variáveis aleatórias na prática, pois quase nunca temos os dados de

[8] https://en.wikipedia.org/wiki/Probability_mass_function

[9] http://www.portalaction.com.br/probabilidades/23-variavel-aleatoria-continua

[10] https://pt.wikipedia.org/wiki/Distribuiç~ao_normal

[11] Basta calcular a integral $\int_{-1}^{2} f(z)\mathrm{d}z$.

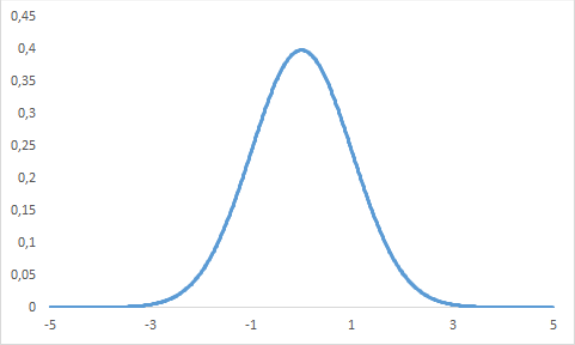


Figura 3.1: Função Densidade

todos os indivíduos de uma população. Geralmente temos somente acesso a uma amostra finita da população, ou seja, uma sequência de realizações de variáveis aleatórias. Cada variáveis aleatória é dotada de diversos valores intrinsecamente associado a elas - são esses: média (ou esperança), mediana, variância, desvio-padrão e covariância. Dado o fato que geralmente estamos trabalhando com amostras finitas da população e não com a população inteira, nós gostaríamos de estimar esses valores associados à distribuição de uma certa variável aleatória.

3.1.5 Média

A Média ou Esperança/Valor Esperado[12] é valor que esperamos se sortearmos um número aleatória que veio de uma distribuição de probabilidade. Se o salário médio do brasileiro é R$1000, por exemplo, dizemos que se escolhermos um brasileiro aleatóriamente, esperamos que seu salário seja de R$1000. Uma das formas de estimar esse valor esperado, é calculando a média aritmética simples dos valores de nossa amostra[13], que é a mais rotineira, sendo usada para vários cáculos estatísticos, que veremos a frente. A média é calculada pela somatório dos valores observados divido pelo número deles. Formalizando o conceito temos que se x_1, x_2, ..., x_n são valores relativos à variável aleatória X, e n é o número de valores realizado a partir de X. Podemos escrever a média da seguinte maneira:

$$\bar{X} = \frac{1}{N}\sum_{i=1}^{N} X_i \tag{3.2}$$

3.1.6 Mediana

Assim como a média/esperança, a mediana é uma medida de "tendência central", ou seja, é um valor no qual os outros dados se aproximam. Por definição, a mediana ocupa a posição central dos dados quando ordenados, ou seja, é o valor que divide a distribuição de probabilidade exatamente ao meio, colando 50% dos valores em cada um dos lados. Estimar a mediana é diferente quando o tamanho da amostra é par ou ímpar. Quando o n é par, realizamos a soma de mais um ao resultante da divisão de n por dois e somamos com do resultado da média de x_1, x_2, ..., x_n e dividimos por dois, assim encontrando a posição da mediana. Para achar o valor da mediana, deve fazer a média dos dois

[12] `https://pt.wikipedia.org/wiki/Valor_esperado`
[13] Esperamos que nossa amostra seja aleatória.

valores próximos a posição encontrada. Caso o n seja ímpar, basta atribuir para à mediana o valor da posição resultante da divisão por dois de $n+1$. A fórmula pode ser definida como:

$$\text{md}(X_1, ..., X_n) = \begin{cases} \frac{N+1}{2} & \text{, se } N = \text{ímpar} \\ \\ \frac{\frac{N}{2}+(\frac{N}{2}+1)}{2} & \text{, se } N = \text{par} \end{cases} \tag{3.3}$$

3.1.7 Variância

Dentre as medidas possíveis de serem calculadas, a variância[14] é considerada uma medida de dispersão, representando quanto, em média, os valores associados a uma distribuição de probabilidade estão distantes da média. Existem basicamente 2 tipos de variância: a Variância Populacional e a Variância Amostral (ou estimada). A Variância Populacional é uma medida de dispersão em toda a população. Como geralmente é impossível ter acesso aos dados da população inteira e calcular essa variância, conseguimos estimá-la com a Variância Amostral. A fórmula da variância amostral para realizações de uma variável X corresponde ao resultado da divisão do somatório das diferenças (ao quadrado) entre os valores relativos à variável X e sua média aritmética pelo número de observações na amostra menos 1:

$$var(X_1, ..., X_n) = S^2 = \frac{\sum_{i=1}^{N}(X_i - \bar{X})^2}{N-1} \tag{3.4}$$

Já a variância populacional é calculada da seguinte forma:

$$var(X_1, ..., X_n) = \sigma^2 = \frac{\sum_{i=1}^{N}(X_i - \bar{X})^2}{N} \tag{3.5}$$

3.1.8 Desvio-Padrão

O fato de a variância ser uma medida que tem escala igual ao quadrado da escala dos dados[15] pode causar grande confusão na sua interpretação. Para solucionar esse problema, costuma-se usar o desvio-padrão, que é a raiz quadrada da variância. Assim como a variância, podemos dividir o Desvio-Padrão e Desvio-Padrão Populacional e Amostral. Para se calcular o Desvio-Padrão Populacional/Amostral basta tirar a raíz quadrada da Variância Populacional/Amostra, respectivamente. A seguir, a fórmula do Desvio-Padrão Amostral:

$$dp(X_1, ..., X_n) = S = \sqrt{var(X_1, ..., X_n)} \tag{3.6}$$

Já o desvio padrão populacional é calculada da seguinte forma:

$$dp(X_1, ..., X_n) = \sigma = \sqrt{var(X_1, ..., X_n)} \tag{3.7}$$

[14] `https://pt.wikipedia.org/wiki/Variância`.

[15] Tomando como exemplo dados expressos em cm, a variância será expressa em cm^2.

3.1.9 Covariância

A medida em que trabalhamos com mais dados, podemos começar a ter valores que medem a relação entre várias variáveis. Em um caso de querer ver se existe uma relação entre a variável X e a variável Y, por exemplo, podemos usar a covariância[16], que mede a relação linear entre as duas variáveis. Assim como a variância, usaremos a covariância amostral para estimar essa relação linear na população. A covariância amostral é o somatório do produto das diferenças dos valores relativos a variável X com a média estimada de X, juntamente com os valores de relativos a variável Y com a média estimada de Y, tudo divido pela quantidade de valores menos um. Considere nossa amostra como sendo o par $(\mathbf{x}, \mathbf{y})$, $\mathbf{x} = (x_1, ..., x_n)$ e $\mathbf{y} = (y_1, ..., y_n)$ como sendo as realização das variáveis aleatórias X e Y:

$$cov(\mathbf{X}, \mathbf{Y}) = S_{XY} = \frac{\sum_{i=1}^{N}(X_i - \bar{X})(Y_i - \bar{Y})}{N-1} \tag{3.8}$$

Já a covariância populacional pode ser calculada da seguinte forma:

$$cov(\mathbf{X}, \mathbf{Y}) = S_{XY} = \frac{\sum_{i=1}^{N}(X_i - \bar{X})(Y_i - \bar{Y})}{N} \tag{3.9}$$

Quanto maior o valor da covariância em módulo, maior o grau de relação linear entre as variáveis. Quando o valor da covariância é baixo em módulo, a relação linear entre as variáveis é mais fraca. O sinal negativo indica que a medida que X, aumenta os valores de Y tendem a diminuir e vice-versa. O sinal positivo indica que se os valores de X aumentam, os valores em Y aumentam também. É muito **importante** frisar que a existência de uma relação linear entre variáveis não implica em uma relação de causalidade.

3.1.10 Coeficiente de Variação

O coeficiente de variação cv, é uma medida de dispersão, usado principalmente quando queremos comparar dois conjuntos de dados que possuem médias diferentes. Sua formula é a razão do desvio padrão pela média. Seu resultado geralmente é interpletado em porcentagem, mostrando a porcentagem da variabilidade dos dados.

$$cv = \frac{dp}{\bar{X}} \tag{3.10}$$

3.1.11 Coeficiente de Correlação de Pearson

O coeficiente de correlação de Pearson, ou coeficiente de correlação porduto-momento [17] é usado para averigar se existe uma relação linear entre os dados, variando de -1 a 1. Onde p = 1 significa que existe uma correlação perfeita entre os valores; p = 0 significa que as duas variáveis não são dependentes uma da outra, ou seja, não há uma correlação linear entre elas; p = -1 significa que os valores possuem uma correlação negativa, ou seja, quando um valor aumenta o outro diminui. A

[16] `https://en.wikipedia.org/wiki/Covariance.`

[17] `https://pt.wikipedia.org/wiki/Coeficiente_de_correlaç~ao_de_Pearson`

fórmula é dada pelo resultado da razão da covariância pelo resultado da multiplicação dos desvios padrões.

$$p = \frac{cov(X,Y)}{\sqrt{var(X) * var(Y)}} \tag{3.11}$$

3.2 Álgebra Linear

A Álgebra Linear é uma área fundamental da matemática com uma importância tão grande quanto à do Cálculo. Os objetos de estudo da Álgebra Linear são os Espaços Vetoriais, Vetores, Matrizes, Transformações Lineares entre outras coisas. Mas qual a importância disso tudo para a análise de dados e *Machine Learning*? Grande parte dos modelos estatísticos e matemáticos que veremos daqui para frente usam os conceitos de Vetores e Matrizes em sua construção, sem falar que muitas vezes conhecê-los pode facilitar a vida um bocado. É importante salientar que o conteúdo que será exposto nesse capítulo tem caráter extremamente prático, então deixaremos as partes mais teóricas para os livros didáticos. Mesmo que já tenha cursado essa disciplina anteriormente, acompanhe o capítulo e faça os exercícios propostos. Se ainda sim não ficar satisfeito, acompanhe nossas sugestões de leitura. Vamos lá!

3.2.1 Matrizes

Matrize[18] são uma coleção[19] de números em forma de "tabela":

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{pmatrix} \tag{3.12}$$

A matriz A tem ordem $3x4$ (3 linhas e 4 colunas), logo dizemos que $A \in \mathbb{R}^{3x4}$. A título de curiosidade, dizemos que $A \in \mathbb{R}^{3x4}$ pertence a um espaço com 12 dimensões, pois os elementos que o habitam tem 12 entradas compostas por números reais. A um exemplo mais palpável de matriz A seria;

$$A = \begin{pmatrix} 0 & 4 & 1 & 7 \\ 9 & 3 & 4 & 1 \\ 6 & 3 & 1 & 0 \end{pmatrix} \tag{3.13}$$

3.2.2 Vetores

Os vetores são objetos que são encontrados nos Espaços Vetorias[20], que são estruturas matemáticas - se você for curioso(a) vale a pena conferir, mas se não tiver paciência, não há grandes problemas. Para nossos fins, vamos assumir que o espaço no qual estaremos trabalhando é o $\mathbb{R}^n$, sendo que n é um número Natural diferente de zero. Ou seja, os elementos que habitam o $\mathbb{R}^n$ são pontos, como os pares

[18] Para mais detalhes: `https://en.wikipedia.org/wiki/Matrix_(mathematics)`
[19] Em computação, essas coleções se chamam *arrays*.
[20] `https://en.wikipedia.org/wiki/Vector_space`

ordenados, mas com n coordenadas. Devido a uma série de definições e resultados matemáticos[21] cada um desses pontos representa um objeto geométrico chamado Vetor[22] e devido a esses resultados matemáticos já citados, trabalhar com os pontos no $\mathbb{R}^n$ é equivalente a trabalhar com os vetores diretamente. Então a partir de agora passaremos a chamar pontos da forma $\mathbf{x} = (x_1, x_2, ..., x_n)$ de vetores. Além dessa forma, o vetor $\mathbf{x}$ também pode ser representado na forma matricial[23]:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{pmatrix} \tag{3.14}$$

3.2.3 Operações com matrizes e vetores

Adição e subtração

Quando queremos somar duas matrizes (incluindo vetores na forma matricial) basta somarmos elemento por elemento. Logo, se $A + B = C$, então $a_{ij} + b_{ij} = c_{ij}$. por outro lado, se $A - B = C$, então $a_{ij} - b_{ij} = c_{ij}$. Pela definição de adição e subtração, temos que as matrizes A e B devem ter a mesma ordem nXm.

Produto por um escalar

Quando queremos multiplicar matrizes (incluindo vetores na forma matricial) por um escalar α, basta multiplicarmos todos os elementos da matriz pelo escalar. Ou seja, se $B = \alpha A$ então, $b_{ij} = \alpha a_{ij}$.

Multiplicação de duas matrizes

Quando queremos multiplicar uma matriz (incluindo vetores na forma matricial) por outra matriz temos que aplicar uma fórmula específica. Vamos supor que $A \in \mathrm{R}^{nxk}$ e $B \in \mathrm{R}^{kxm}$ então se $C = AB$, logo $c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + ... + a_{ik}b_{kj} = \sum_{p=1}^{k} a_{ip}b_{pj}$. Da nossa definição seguem as seguintes conclusões: (i) o número de colunas da primeira matriz (A) deve ser igual ao número de linhas da segunda (B) e (ii) e matriz produto C tem o número de linhas da primeira e o número de colunas da segunda.

Produto elemento a elemento

Nesse caso necessitamos que as duas matrizes que estão sendo multiplicadas tenham a mesma ordem. Se $C = A \bigodot B$ então $c_{ij} = a_{ij}b_{ij}$.

Transposição

A transposição é uma operação que inverte as colunas e as linhas de uma matrix, ou seja, se $B = A^T$ é a transposta de A, então $b_{ij} = a_{ji}$.

Links externos

Para ver com mais detalhes como funcionam as operações com matrizes, basta acessar os seguintes links:

- `https://www.georgebrown.ca/uploadedFiles/TLC/_documents/Basic%20Matrix%20Operations.pdf`

[21] Base ortonormal, coordenadas e espaços isomorfos.
[22] Mais tarde veremos os vetores na forma matricial.
[23] A convenção é de adotar vetores coluna - com n linhas e 1 coluna.

- https://www.infoescola.com/matematica/operacoes-com-matrizes-1/
- https://en.wikipedia.org/wiki/Matrix_(mathematics)

3.2.4 Matrizes especiais

1. Matriz simétrica: a matriz simétrica é aquela que satisfaz $A = A^T$. Deve ser quadrada, ou seja, o número de linhas deve ser igual ao número de colunas;
2. Matriz identidade: a matriz identidade é tem zeros em todas suas entradas, exceto pela sua diagonal principal, que é composta exclusivamente por uns. Deve ser quadrada e é denotada pela letra "i"maiúscula - I. A matriz I_n, por exemplo, é uma matriz identidade de ordem nXn. Uma propriedade de I é que se $AI_m = I_nA = A$, para qualquer matriz $A \in \mathrm{R}^{nxm}$;
3. Matriz inversa: se A^{-1} é a inversa de A, então $A^{-1}A = AA^{-1} = I$.
4. Matriz de co-ocorrência são matrizes quadradas, geralmente utilizadas em imagens, onde são utilizadas para o cálculo de descritores. Nessas matrizes são armazenadas os valores de níveis de cinza da imagem nos seguintes ângulos: $0, 45, 90, 135$, os demais ângulos são calculados por simetria [24].
5. Matriz de covariância: são as matrizes que contém as variâncias das variáveis em sua diagonal e nos demais pontos da matriz possui a covariância de todas as combinações dos pares das variáveis. São caracterizadas por serem matrizes quadradas. Geralmente são empregadas em aplicações estatísticas.

3.2.5 Produto escalar e norma euclidiana

Se ambos $\mathbf{x} = \begin{pmatrix} x_1 & x_2 & \dots & x_n \end{pmatrix}$ e $\mathbf{y} = \begin{pmatrix} y_1 & y_2 & \dots & y_n \end{pmatrix}$ são elementos do conjunto R^n então o produto escalar, ou muitas vezes referenciado como produto interno ou *inner product*, desses dois vetores é definido como $\sum_{i=1}^{n} x_i y_i = \mathbf{x}^T\mathbf{y}$. A norma euclidiana também conhecida como "módulo"do vetor pode ser calculada com a seguinte fórmula $\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2} = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{\mathbf{x}^T\mathbf{x}}$.

3.2.6 Distância entre vetores

Se ambos $\mathbf{x} = \begin{pmatrix} x_1 & x_2 & \dots & x_n \end{pmatrix}$ e $\mathbf{y} = \begin{pmatrix} y_1 & y_2 & \dots & y_n \end{pmatrix}$ são elementos do conjunto R^n então uma distância entre $\mathbf{x}$ e $\mathbf{y}$ é uma função $d : \mathrm{R}^n \; X \; \mathrm{R}^n \to \mathrm{R}$ tal que (i) $d(\mathbf{x},\mathbf{y}) \geq 0$, (ii) $d(\mathbf{x},\mathbf{y}) = d(\mathbf{y},\mathbf{x})$, (iii) $d(\mathbf{x},\mathbf{y}) \leq d(\mathbf{x},\mathbf{z}) + d(\mathbf{z},\mathbf{y})$ para $\mathbf{z} \in \mathrm{R}^n$ e (iv) $d(\mathbf{x},\mathbf{y}) = 0$ se e somente se $\mathbf{x} = \mathbf{y}$. Observe que dizer $\mathbf{x} = \mathbf{y}$ é igual a dizer que $x_i = y_i$ para qualquer i. Vamos agora introduzir as distâncias mais importantes para a nossa evolução no conteúdo:

Distância *Manhattan* ou *Taxicab*

Dados $\mathbf{x},\ \mathbf{y} \in \mathrm{R}^n$, a Distância *Manhattan*[25] é $L = \sum_{i=1}^{n} |x_i - y_i|$.

Distância Euclidiana

Dados $\mathbf{x},\ \mathbf{y} \in \mathrm{R}^n$, a Distância Euclidiana[26] é $L^2 = \|\mathbf{x} - \mathbf{y}\| = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} = \sqrt{(\mathbf{x} - \mathbf{y})^T (\mathbf{x} - \mathbf{y})}$.

[24] http://iris.sel.eesc.usp.br/wvc/Anais_WVC2012/pdf/97967.pdf

[25] https://en.wiktionary.org/wiki/Manhattan_distance

[26] https://en.wikipedia.org/wiki/Euclidean_distance e http://mathonline.wikidot.com/the-distance-between-two-vectors

Distância de Mahalanobis

Dados $\mathbf{X},\ \mathbf{Y} \in \mathrm{R}^n$ vetores aleatórias e $\mathbf{x}$ e $\mathbf{y}$ realizações dos mesmos, a Distância de Mahalanobis[27] empírica é dada por $D_M = D_M(\mathbf{x},\mathbf{y}) = \sqrt{(\mathbf{x}-\mathbf{y})^T S^{-1}(\mathbf{x}-\mathbf{y})}$, sendo que S é a matriz estimada de covariância entre $\mathbf{X}$ e $\mathbf{Y}$.

Exercício

Se $\mathbf{x} = (1,5)$, $\mathbf{y} = (2,-4)$, $S_X = 2$ (desvio-padrão), $S_Y = 1$ (desvio-padrão) e $S_{XY} = 3$ (covariância), calcule:

- L
- L^2
- D_M

3.2.7 *Machine Learning - K Nearest Neighbours* para classificação

O algoritmo *K Nearest Neighbours* é um importante algoritmo de classificação no campo de *Machine Learning*, que apesar de sua simplicidade continua sendo muito poderoso. Vamos começar com uma explicação um pouco mais teórica, mas logo vamos ao código para trabalharmos com dados reais. Primeiramente, é preciso introduzir algumas ideias centrais do trabalho com algoritmos de *Machine Learning*.

Intuição

Suponha que você tenha uma base de dados de países e que estes estejam divididos em dois grupos: países em desenvolvimento (roxos) e países desenvolvidos (amarelos). Essa classificação foi feita com base em uma metodologia e dados de um órgão internacional e você é incapaz de reproduzir os resultados perfeitamente, pois não tem acesso a essas informações. No entanto, você percebeu que duas variáveis são capazes de informar muito a respeito da situação dos países: **índice de criminalidade** (X_2) e **índice de educação** (X_1). Temos quanto menor o índice de criminalidade e maior o índice educacional mais desenvolvido o país tende a ser. Apesar de tudo isso, você acabou de incluir o país Estrela em sua base de dados com as informações de educação e criminalidade. No entanto, esse país ainda não está classificado como desenvolvido ou em desenvolvimento e você gostaria de *predizer* qual seria a classificação do órgão com base nas variáveis disponíveis.

A estratégia a partir desse momento será de calcular a distância[28], que é uma medida de similaridade, entre o país Estrela e os demais países. Depois de calcular as distância, basta definir como faremos a classificação a partir da escolha do número K, que deve ser um valor real diferente de zero - se $K = 10$, por exemplo, pegamos os 10 países mais similares ao Estrela e, dentre esses 10, a classe de países que for mais frequente será a nossa predição. Ou seja, se for 10 países, 8 forem desenvolvidos, o país Estrela será classificado como desenvolvido, pois ela está mais próximo desses países. Se o número for exatamente igual à metade, não há uma boa predição, logo tanto faz a classe predita para o país Estrela, desenvolvido ou em desenvolvimento.

É possível perceber pela Figura 3.2.7 que se $K = 3$, o país Estrela é classificado como país em desenvolvimento (roxo) e se $K = 6$, o país Estrela é classificado como país desenvolvido (amarelo). Pelo o que vimos, o número K é um parâmetro um tanto quanto arbitrário e pode ter grande influência no resultado da predição. Por isso dizemos que K é um Hiperparâmetro ou *Hyperparameter*. Mas como escolher K? Veremos mais à frente.

[27] `https://en.wikipedia.org/wiki/Mahalanobis_distance`

[28] A princípio, pode ser qualquer uma das que vimos. No entanto, estaremos adotando a Euclidiana no exemplo, por ser mais utilizada.

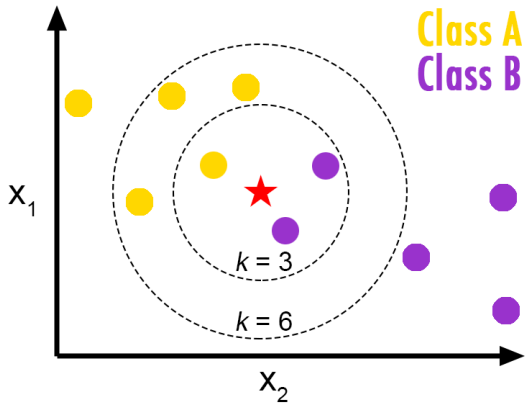


Figura 3.2: Classificando o novo país

A estrutura dos Dados, *Training Set* e *Test Set*

O algoritmo de *KNN* é conhecido por ser um método de classificação *Supervisionado*, ou seja, o computador "aprende"com exemplos de classificações já realizadas para prever resultados futuros e fazer classificações de observações sem *label*. No nosso exemplo, o país Estrela estava sem *label*, ou seja, não estava classificado e usamos observações já classificadas para fazer predizer sua classificação com base nas distâncias. Geralmente quando estamos trabalhando com a uma base de dados, em uma situação ideal, todos as nossas observações já estão com *label* e o nosso principal objetivo é fazer a classificação de observações que ainda não estão presentes nos dados, mas entrarão no futuro sem o *label*. Por esses motivos, devemos simular uma situação em que temos certas observações não classificadas para testar a eficácia do algoritmo para executar a tarefa.

Quando estamos trabalhando com métodos de *Aprendizagem Supervisionada*, é comum dividir a base de dados que temos em duas partes: (i) *Training Set* e (ii) *Test Set*. O *Training Set* comporta a grande maioria da base de dados (de 80% a %90 das observações), com o intuito de "treinar' o modelo (no caso, KNN) na realização das tarefas. O *Test Set* comporta o restante da base de dados e tem o intuito de verificar a acurácia do modelo treinado no *Training Set* - no caso da classificação, é necessário comparar as classificações originais com as classificações feitas pelo modelo. É importante deixar claro que um bom desempenho do modelo no *Training Set* não implica um bom desempenho no *Test Set*, pois ainda corremos o perigo de o modelo "decorar"a estrutura dos dados utilizados no treinamento e, por conta disso, ter uma predição prejudicada quando se trata de observações nunca vistas pelo modelo.

Hyperparameters, *Validation Set* e a escolha do K

Hiperparâmetros são aqueles parâmetros arbitrários irão influenciar no nosso modelo, sendo assim, deve-se escolher o valor dos mesmos *a priori*. No caso do *KNN*, o Hiperparâmetro de nosso interesse é o valor de *K*, que é o número de vizinhos mais próximos usados na hora de se fazer a classificação. Quando se trata de *Machine Learning*, os valores assumidos pelos Hiperparâmetros são decididos em um processo chamado *Hyperparameterstuning*: treinamento e testamos o modelo para diversos valores para os Hiperparâmetros, escolhendo assim, o conjunto de valores que tiveram melhor desempenho. Para fazer a escolha do conjunto de valores para os Hiperparâmetros é necessário aplicar o conceito de *Validation*, ou seja, precisamos passar nosso modelo por um processo de validação. Para realizar esse processo, basta selecionar um subconjunto *Training Set* (por volta

de 5% a 10% do conjunto de treinamento), e testar modelos com diferentes Hiperparâmetros. Por exemplo, se destinarem 10% da base original para o *Test Set* e 5% do conjunto de treinamento para a validação, teremos relativamente à base original: 85,5% para treinamento, 4,5% para validação e 10% para teste, No fim das contas, a parte do *Training set* que foi destinada à validação não foi usada para o treinamento do modelo.

Preparação dos dados

Existem dois grandes entraves na hora de se utilizar o modelo KNN na prática: (i) a escala e variação das variáveis contínuas e (ii) a existência de variáveis categóricas. Aqui vamos somente falar sobre o problema (i) por algumas razões: essa apostila tem caráter introdutório e KNN, definitivamente, não é o método mais aconselhável para se trabalhar com variáveis categóricas. No entanto. se tiver curiosidade, fazer visitar os seguintes conteúdos: Conteúdo 1 e Conteúdo 2. Logo assumimos que todas as variáveis que usaremos para calcular as distâncias são contínuas.

Por que a escala e a variação das variáveis pode ser considerado um problema? Por um motivo simples. Imagine estamos calculando a distância entre dois vetores usando duas variáveis, sendo que a primeira tem média 0 e desvio-padrão 1 e a segunda tem média 1000 e desvio padrão 100. Como queremos sempre encontrar pontos que próximos àqueles que queremos classificar, a classificação, nesse caso, não utilizará todo o potencial contido na segunda variável, pois qualquer variação nesta segunda variável vai acarreta um grande acréscimo na distância. Por exemplo, considere $\mathbf{x} = (0,1000)$, $\mathbf{y} = (1,900)$ e $\mathbf{z} = (10,901)$. Comparando as Distâncias Euclidianas temos que $d(\mathbf{x},\mathbf{y}) > d(\mathbf{x},\mathbf{z})$. Esse resultado faz sentido numericamente falando, mas parece não ser razoável, pois no eixo da primeira variável, **x** e **y** estão há 1 desvio-padrão, enquanto **x** e **z** estão há 10 desvios-padrão de distância. Por outro lado, quando focamos no eixo da segunda variável, a diferença é de apenas 10% de um desvio-padrão e, mesmo assim, **y** parece estar mais distante de **x** comparativamente à **z**.

Existem diversas maneiras de contornar esse problema e aqui vamos apresentar as duas mais comuns. As duas maneiras implicam em transformações que devem ser aplicadas em todas as variáveis que estamos trabalhando. A primeira delas é a Normalização entre 0 e 1, ou seja, todas as variáveis são normalizadas para caberem nesse intervalo. Se temos uma variável X, após a Normalização:

$$X_{norm.} = \frac{X - \min(X)}{\max(X) - \min(X)} \tag{3.15}$$

Sendo que $\min(X)$ é o valor mínimo que a variável X assume no **Training set** e $\max(X)$ é o valor máximo. É importante lembrar que **training set**, nesse caso, não inclui a porção usada no Processo de Validação. A outra maneira de tratar os dados antes de aplicar a técnica do KNN é chamada de Padronização das variáveis. Com essa padronização, todas as variáveis passam a ter média zero e desvio-padrão igual a 1. É necessário frisar que assim como $\min(X)$ e $\max(X)$, as estatísticas $\bar{X}$ e S_X devem ser calculadas no **Training set**.

$$X_{padr.} = \frac{X - \bar{X}}{S_X} \tag{3.16}$$

Para entender melhor como todo esse processo funciona, **vamos ao código**!

3.2.8 Materiais complementares

Para ter uma melhor noção de Álgebra Linear com aplicações em Python, acesse: http://www.ulaff.net/. Para entender um pouco mais a Lógica do Machine Learning `http://www.deeplearningbook.org/contents/ml.html`. Outros materiais de apoio para entender melhor KNN:

- `https://kevinzakka.github.io/2016/07/13/k-nearest-neighbor/`
- `http://cs231n.github.io/classification/#nn`
- `https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm`
- `https://saravananthirumuruganathan.wordpress.com/2010/05/17/a-detailed-introduction-to-k`

3.3 Otimização de funções

3.3.1 O conceito de Derivadas

Vocês já pararam para se perguntar qual é o significado do conceito de Velocidade e por que esta tem sempre uma unidade de espaço dividida por uma de tempo? Velocidade não é nada mais que uma medida de variação de um corpo no espaço dividida por uma medida na variação de tempo que o corpo tomou para se deslocar. Quando a variação de tempo é uma constante maior do que zero, se aplicarmos a definição de velocidade exposta acima, teremos a velocidade média do corpo durante seu percurso em um determinado intervalo de tempo, logo $V_m = \frac{\Delta y}{\Delta x}$. Para facilitar o raciocínio, estamos assumindo que o corpo viaja em uma linha reta ao longo do eixo y e que o eixo x é o tempo. Na Figura 3.3.1 [29] é possível ver a situação de forma ilustrada.

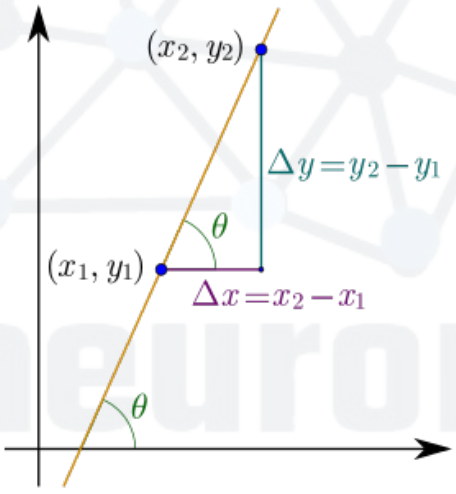


Figura 3.3: Função deslocamento 1

Pela nossa definição de Velocidade Média, esta é dada pela tangente do ângulo θ, ou seja, $V_m = \frac{\Delta y}{\Delta x} = \text{tg}(\theta)$. Nossa função de deslocamento do veículo na Figura 3.3.1 é linear e positivamente inclinada, ou seja, a velocidade com a que o carro se move é constante e positiva $\rightarrow \Delta y$ e Δx têm sempre sinais iguais. O caso da Figura 3.3.1 é um tanto restrito, pois estamos tratando de uma função linear. Por que não partimos para uma função não-linear agora? Vamos lá. na Figura 3.3.1 [30], a curva azul denota nossa função deslocamento e a curva roxa uma reta secante "cortando"a função nos

[29] `https://en.wikipedia.org/wiki/Derivative`.
[30] `https://en.wikipedia.org/wiki/Derivative`.

pontos $(x_1, y_1) = (x, f(x))$ e $(x_2, y_2) = (x+\Delta x, f(x+\Delta x)) = (x+h, f(x+h))$. Logo a velocidade média do carro durante esse deslocamento é dada pelo coeficiente angular (*slope*) da reta roxa. Em outras palavras, a tangente do menor ângulo formado entre a reta roxa e o eixo x é a velocidade média no trecho $f(x) \to f(x+h)$.

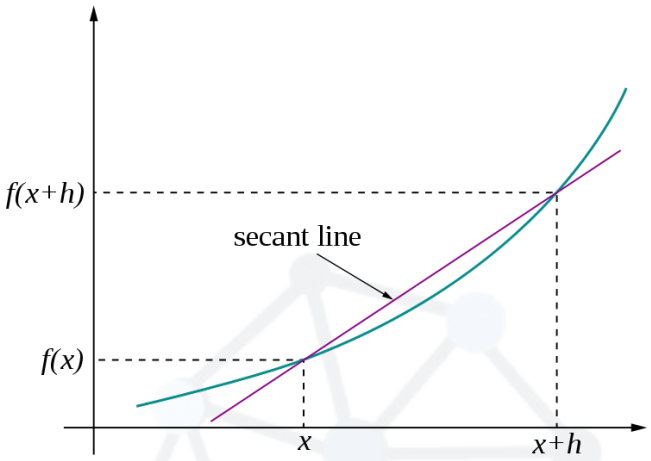


Figura 3.4: Função deslocamento 2

Aí surge uma outra pergunta muito interessante: e quando olhamos para o velocímetro do carro? O que exatamente aquilo está no dizendo? E a resposta é: o velocímetro nos dá a velocidade instantânea do carro e não a velocidade média. Ou seja, o intervalo de tempo (Δx ou h) que o carro utiliza para calcular a velocidade mostrada no velocímetro é tão pequeno que dizemos que este é **infinitesimal** ou infinitesimalmente pequeno[31]. Quando temos um $\Delta x = h$ infinitesimalmente pequeno, que **tende** a zero, a reta, que antes era secante, agora é tangente à função em x (imagem) e seu coeficiente angular dá a velocidade que o velocímetro indica no momento. Veja a Figura 3.3.1:

As notações mais usuais para denotar a inclinação (coeficiente angular) da reta tangente a uma função $f(.)$ quando $x = x_0$ são $f'(x_0)$ e $\frac{\mathrm{d}f(x_0)}{\mathrm{d}x}$. Na segunda notação dx faz o papel do Δx e o $df(x_0)$ faz o papel do $\Delta f(x)$, ambos quando Δx é infinitesimal. Essa inclinação é nada mais do que a derivada no ponto $x = x_0$. Assim sendo, a derivada tem dois grandes significados:

- Geométrico: a inclinação da reta tangente em um certo ponto;
- Abstrato: denota a relação entre duas grandezas. No caso do carro, a pergunta seria "**o que acontece com a posição do carro quando o tempo passa mais um pouquinho (relativamente ao tempo passado)?"**

Se a derivada no ponto $x = x_0$ é 4, ou seja, $\frac{\mathrm{d}f(x_0)}{\mathrm{d}x} = 4$, para cada unidadezinha de tempo passada, o carro se move 4 vezes mais, relativamente à essa unidadezinha, para frente. Se $\frac{\mathrm{d}f(x_0)}{\mathrm{d}x} = -4$, a interpretação é a mesma, no entanto, os deltas têm sinal contrários e o carro está indo para trás. Agora, se $\frac{\mathrm{d}f(x_0)}{\mathrm{d}x} = 0$, quer dizer que o carro está momentaneamente parado (provavelmente fazendo uma conversão). Se eu fizer o cálculo da derivada para um x genérico, teremos o que se chama

[31] Essa ideia de número infinitesimal está intimamente ligada ao conceito de Limite - `https://en.wikipedia.org/wiki/Limit_(mathematics)`.

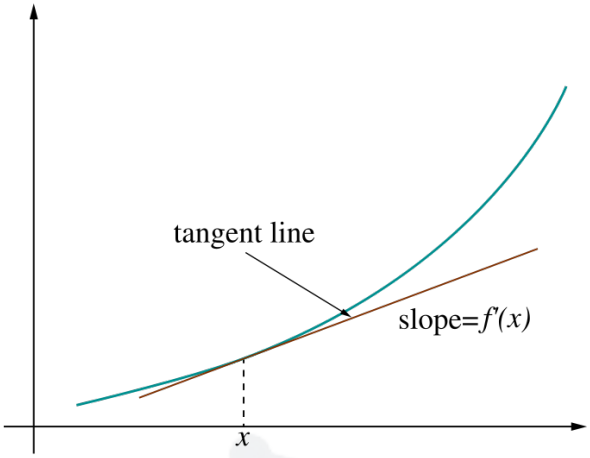


Figura 3.5: Função deslocamento 3

de **função derivada**. Existem técnicas para se derivar uma função $f(x)$ e você **deve** saber as mais básicas. Para isso basta assistir os vídeos 6, 7, 8, 9 e 10 deste curso de derivadas (clique aqui)[32].

Um exemplo aqui é muito importante: vamos assumir $f(x) = x^2$. Pela "regra do tombo", nossa função derivada é $f'(x) = \frac{\mathrm{d}f(x)}{\mathrm{d}x} = 2x$. Quando $x = -2$ temos $f'(-2) = -4$, $x = 0$ implica em $f'(0) = 0$ e $x = 2$ implica em $f'(2) = 4$. Ou seja, quando $x = -2$ a função é decrescente (derivada negativa) e quando $x = 2$ a função é crescente (derivada positiva). Por outro lado, quando $x = 0$ a função não é nem decrescente e nem crescente. Nesse caso, dizemos que $x = 0$ é responsável pelo ponto ponto crítico $(0, f(0)) = (0,0)$ (quando a derivada é **nula**). Observe que se $f(x)$ é diferenciável [33], ser ponto crítico é condição necessária para um ponto ser um ponto de máximo ou de mínimo (onde há uma "conversão"[34]). Por exemplo, se uma função diferenciável $g(x)$ é o custo de uma empresa, quando a empresa minimiza seu custo, temos que $g'(x) = 0$. Geralmente, quando queremos encontrar o mínimo ou máximo de uma função diferenciável, a derivamos e a igualamos a zero, caindo em uma equação. Resolvendo a equação, chegamos a possíveis valores de x que otimizam a função, a partir daí, testamos os valores possíveis de x e descobrimos qual estamos procurando.

Exercícios

Descubra $f'(x)$ para as seguintes funções:

- $f(x) = 2x^4$, Resposta: $f'(x) = 8x^3$
- $f(x) = 2xe^x$, Resposta: $f'(x) = 2(e^x + xe^x)$
- $f(x) = e^{6x}$, Resposta: $f'(x) = 6e^{6x}$

3.3.2 Otimizando uma função na prática

Suponha que queremos minimizar a função $f(x) = 2 + (x-5)^2$ para qualquer x no conjunto dos Reais. Seguiremos, então, os passos apresentados anteriormente: (i) derivar e igualar a zero, (ii)

[32] Você pode baixar a uma tabela de derivadas aqui `https://drive.google.com/file/d/0B1hOECdZUa2xMjA1elMxY1BsbFk/view`

[33] Se tiver curiosidade ver `https://en.wikipedia.org/wiki/Derivative`. Se não tiver, não é muito necessário, dado que as funções vistas no *Machine Learning são diferenciáveis.*

[34] Ou a função está decrescente e começa a ser crescente ou vice-versa

resolver a equação e testar os possíveis candidatos e ver qual dá o mínimo. Derivando $f(x)$ igualando a zero, temos que $f'(x) = 2(x-5) = 0 \Rightarrow x^* = 5$ é o nosso ponto de mínimo. Como é possível perceber, tivemos apenas um candidato e isso ocorreu pois a função $f(x)$ tem pelo menos um ponto de mínimo e é **convexa**[35] em todo seu domínio. Mas e se tivéssemos que resolver esse problema no Python, como seria?

O Método do *Gradient Descent* para a minimização de funções

Para resolver esse problema no Python, até certa parte do caminho o raciocínio é idêntico. Primeiramente, temos que calcular a derivada usando uma caneta e papel - no caso de $f(x)$, sabemos que sua derivada $f'(x) = 2(x-5)$. Em segundo lugar aplicamos o método do *Gradient Descent* (Gradiente Descendente)[36]. Este método é bem simples e segue o seguinte roteiro: (i) escolher um x_0 inicial[37], (ii) calcular a derivada no ponto $x = x_0$, (iii) fazer o *update* do x com a seguinte fórmula $x_{n+1} = x_n - \gamma f'(x_n)$ e repetir o processo até que a sequência $\{x_n\}$ tenha convergido. Aqui não estamos tão preocupados com a convergência no processo e adotaremos um número de repetições grande para garantir que isso aconteça. O γ na nossa fórmula é um número arbitrário e pequeno (e.g. $0,001$) mais conhecido na linguagem do *Machine Learning* como *Learning Rate*, ou seja, é a taxa que o algoritmo usa para "aprender"os padrões nos dados.

Antes de entrar mais a fundo no exemplo, preste muita atenção na Figura 3.3.2. O gráfico nos mostra que à esquerda do $x = 5$ a função é decrescente e à sua direita a função é crescente. É importante relembrar que a função ser decrescente em certo intervalo é o mesmo que dizer que sua derivada é negativa. Por outro lado, dizer que é crescente dá na mesma que dizer que a função tem derivada positiva.

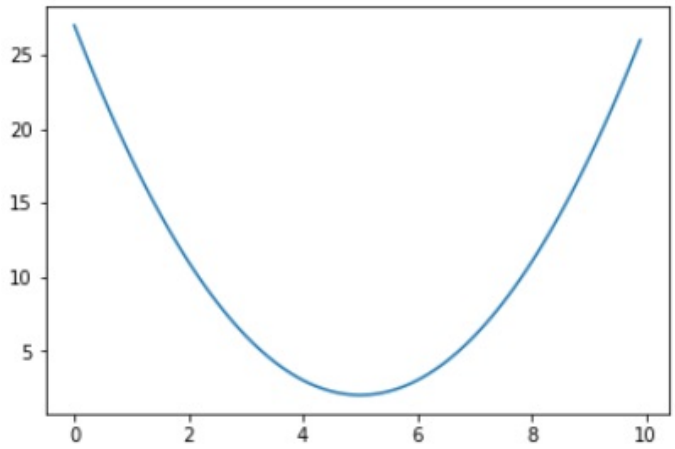


Figura 3.6: $f(x) = 2 + (x-5)^2$

Vamos então seguir os passos para a minimização da função pelo método do *Gradient Descent*: escolhemos um $x_0 = 2$ e uma *Learning Rate* $\gamma = 0,001$ para começar. Em seguida, é necessário aplicar a fórmula para o *update* do nosso x: $x_1 = x_0 - \gamma f'(x_0) = 2 - 0,001(-6) = 2,006$. Seguindo a mesma lógica $x_2 = x_1 - \gamma f'(x_1) = 2,006 - 0,001(-5,988) \approx 2,012$. Perceba que quando vou atualizando o x, este vai cada vez mais se aproximando do nosso x ótimo, ou seja, o 5. Perceba

[35] `https://en.wikipedia.org/wiki/Convex_function`

[36] `https://en.wikipedia.org/wiki/Gradient_descent.`

[37] Geralmente randômico e pequeno.

também que conforme o x vai se aproximando de 5, o valor absoluto da derivada $f'(x)$ vai se aproximando do zero. Assim sendo, quando o x estiver perto o suficiente de 5, a atualização pela fórmula $x_{n+1} = x_n - \gamma f'(x_n)$ vai deixando de surtir efeito. Quando isso ocorre, dizemos que o x **convergiu** para seu valor ótimo, ou seja, o valor que nesse caso minimiza a função. É importante que você se convença que se começássemos com um $x > 5$, por exemplo, $x = 10$ a mesma coisa ocorreria. A única diferença é que o x estaria se aproximando de 5 pela direita e não pela esquerda.

Falta decidir apenas uma coisa para o nosso algoritmo (processo) de minimização - quantas vezes faremos esse *update*. O número de repetições é arbitrário e é conhecido na linguagem de *Machine Learning* como *steps* ou *epochs*. Como padrão nos nossos exemplos, vamos trabalhar com *epochs*= 10000. No entanto, isso não é uma regra e ao longo do seu percurso de aprendizado, você irá aprender melhor como decidir que numero usar e algumas implicações dessa escolha. **Vamos ao código!!! :)**

Exemplo/Exercício 1

Faça o download do Jupyter Notebook e leia o Exemplo 1. Por favor, resolva o Exercício 1.

3.3.3 Otimização de funções com mais de uma variável real

Até o momento estamos trabalhando com funções univariadas (com uma variável real) da forma $f(x)$. Agora começaremos a trabalhar com funções de várias variáveis reais, algo da forma $f(x_1, x_2, ..., x_n)$. Um exemplo de função do tipo seria $f(x,y) = x^2 + y^2$ - nesse caso específico, o domínio da nossa função é um conjunto de pares ordenados, como por exemplo o $\mathbb{R}^2$, que é o conjunto de pares ordenados (x,y) sendo que x e y são números reais.

Derivadas Parciais e Vetor Gradiente

O raciocínio da otimização de funções de várias variáveis reais é análogo ao raciocínio que desenvolvemos para funções univariadas, no entanto, devemos fazer adaptações para caso de maior dimensão. Primeiramente, é necessário ver o conceito de **Derivada Parcial**[38] e para isso devemos relembrar do significado abstrato da derivada no caso univariado: no caso univariado a derivada nos diz a resposta de uma variável y a uma pequena variação de uma variável x. No caso das Derivadas Parciais, a interpretação é a mesma: como um exemplo, tome $z = f(x,y) = x^2 + y^2 + xy$ sendo que estamos interessados no efeito que uma pequena variação na variável y tem sobre a variável z, mantendo x constante. Nesse caso, basta derivar parcialmente z em relação a y - as regras de derivação são as mesmas do caso univariado considerando x como uma constante. O resultado[39] desse cálculo seria $\frac{\partial z}{\partial y} = \frac{\partial f(x,y)}{\partial y} = 0 + 2y + 1x = 2y + x$.

Analogamente ao caso univariado, em uma situação de optimalidade, é necessário que todas as derivadas parciais, ou seja, derivadas em relação a todas as variáveis, sejam iguais a zero. No caso em que $f(x,y) = x^2 + y^2$, por exemplo, a situação de optimalidade exige que $\frac{\partial f(x,y)}{\partial x} = \frac{\partial f(x,y)}{\partial y} = 0$, pois a função $f(x,y)$ é diferenciável. Ou seja, $2y = 2x = 0 \Rightarrow x^* = y^* = 0$, logo o resultado ótimo e que dá o valor mínimo da função f é o ponto $(0,0)$. Para facilitar as contas e os algoritmos futuros, introduziremos agora o conceito de Vetor Gradiente, que é o lugar da onde saiu o nome do nosso algoritmo *Gradient Descent*.

O Vetor Gradiente é um vetor que está relacionado a uma função multivariada da forma $f : \mathbb{R}^n \to \mathbb{R}$ e sempre terá a ordem n. Se $f = f(x_1, x_2, ..., x_n)$ então o Vetor Gradiente de f

[38] `https://pt.wikipedia.org/wiki/Derivada_parcial`

[39] Se não entendeu o resultado do cálculo, favor voltar aos materiais indicados para as regras de derivação.

é $\nabla f = \nabla f(x_1, x_2, ..., x_n) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, ..., \frac{\partial f}{\partial x_n}\right)$ ou na forma matricial $\nabla f = \nabla f(x_1, x_2, ..., x_n) = \left(\frac{\partial f}{\partial x_1} \ \frac{\partial f}{\partial x_2} \ ... \ \frac{\partial f}{\partial x_n}\right)^T$. Se f é definida como $f(x,y) = x^2 + y^2$, logo o Vetor Gradiente de f é $\nabla f = \nabla f(x,y) = \left(\frac{\partial f}{\partial x} \ \frac{\partial f}{\partial y}\right)^T = (2x \ \ 2y)^T$. Logo a condição necessária para a optimalidade, para funções bem comportadas, é de que o vetor gradiente seja nulo.

Exercícios

Descubra ∇f para as seguintes funções:

- $f(x,y) = 2yx^4$
- $f(x,y) = 2xe^{xy}$
- $f(x,y,z) = xyz + ze^{6xy}$

Adaptando o método do *Gradient Descent* para o caso multivariado

A adaptação do método de minimização de funções utilizado até agora é simples. Considere que estamos trabalhando com uma função bivariada $f(x,y)$[40], aí então precisamos escolher um ponto inicial (x_0, y_0) e depois aplicar o *update* sucessivamente para que nosso par (x,y) convirja para o ponto ótimo (x^*, y^*). A regra do *update* no caso multivariado fica $(x_n, y_n) = (x_{n-1}, y_{n-1}) - \gamma \nabla f(x_{n-1}, y_{n-1})$. Se você prestar bem a atenção, o caso univariado não nada mais ou menos do que um caso particular.

Vamos dizer então que $f(x,y) = x^2 + y^2$ e que escolhemos uma *Learning Rate* $\gamma = 0,001$ e um ponto inicial $(x_0, y_0) = (-2, 3)$. Logo $(x_1, y_1) = (x_0, y_0) - \gamma \nabla f(x_0, y_0) = (-2, \ 3) - 0,001(-4, \ 6) = (-1.996, \ 2.994)$ e, seguindo a mesma lógica, $(x_2, y_2) = (x_1, y_1) - \gamma \nabla f(x_1, y_1) = (-1.996, \ 2.994) - 0,001(-3.992, \ 5.988) \approx (-1.992, \ 2.988)$. Como vimos anteriormente, o ponto que minimiza a função nesse caso é $(0,0)$ e conforme os *updates* vão sendo feitos a sequência de pontos $\{(x_n, y_n)\}$ se aproxima do ponto ótimo. É importante salientar que, assim como no caso univariado, conforme vamos nos aproximando do ponto ótimo, o Vetor Gradiente vai se aproximando do vetor nulo, até que os *updates* deixam de surtir efeito e dizemos que houve convergência. **Vamos ao código!**

Exemplo/Exercício 2

Faça o download do Jupyter Notebook e leia o Exemplo 2. Por favor, resolva o Exercício 2.

3.3.4 *Machine Learning* - Ajuste de curvas entre pontos

O ajuste de curvas em nuvens de pontos é um velho problema da matemática e da estatística. A resolução desse problema se dá pelo método dos Mínimos Quadrados que é a base para o conceito de **Regressão Linear**. Suponha que você tenha a seguinte situação: um(a) vendedor(a) de sorvetes percebe que a venda de sorvetes está positivamente relacionada à temperatura média do dia. Após ter coletado os dados de temperatura média (eixo x) e quantidade de sorvetes vendidos (eixo y) chegamos ao seguinte gráfico (3.3.4):

Um(a) cientista de dados quer saber agora qual a função linear da forma $f(x) = wx$ que melhor descreve essa relação. No entanto, a constante w é desconhecida. Para isso, o(a) especialista em análise de dados propõe o método dos Mínimos Quadrados para a estimação do parâmetro w. Nossa amostra é composta pelos pares $(20,50), (22,65), ..., (35,130)$, sendo que temos 7 pontos (pares) no total. Logo, nossa amostra pode ser reescrita da seguinte maneira $(x_0, y_0), (x_1, y_1), ..., (x_6, y_6)$. O método dos Mínimos Quadrados se consiste em encontrar o valor de w que **minimiza** o Erro

[40] Para se trabalhar com N variáveis é só estender o raciocínio.

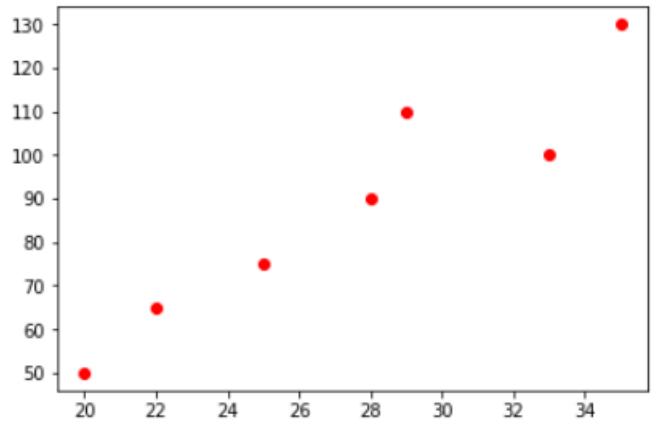


Figura 3.7: Temperatura Média Vs. Venda de sorvetes no dia

Quadrático Médio (EQM). No nosso caso, o EQM é uma função de w e é dado pela equação 3.17:

$$\text{EQM}(w) = \frac{1}{7}(y_0 - wx_0)^2 + \ldots + \frac{1}{7}(y_6 - wx_6)^2 = \frac{1}{7}\sum_{i=0}^{6}(y_i - wx_i)^2 = \frac{1}{7}(\mathbf{y} - w\mathbf{x})^T(\mathbf{y} - w\mathbf{x}) \quad (3.17)$$

Se **y** e **x** forem vetores que contém as amostras. Pela resolução conseguimos traçar uma reta entre os pontos com uma equação igual a $y = 3.28x$. Veja na Figura 3.3.4:

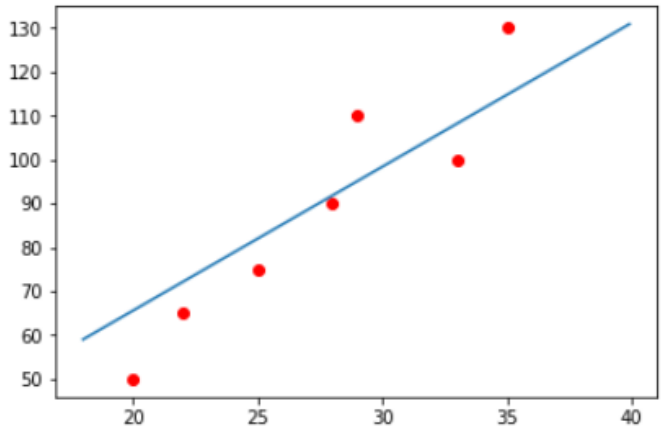


Figura 3.8: Temperatura Média Vs. Venda de sorvetes no dia

Para aplicarmos o Método de *Gradient Descent* e achar o w que minimiza o EQM, basta seguir os passos que repetimos diversas vezes anteriormente!

Exemplo/Exercício 3

Faça o download do Jupyter Notebook e leia o Exemplo 3. Por favor, resolva o Exercício 3.

3.3.5 Observações finais

Nos exemplos apresentados durante essa parte de otimização de funções focamos praticamente na minimização de funções. Fizemos isso pois na maioria das aplicações que temos em *Machine Learning* precisamos minimizar funções ao invés de maximizar. No entanto, se houver a necessidade de se maximizar uma função, é possível fazê-lo com pequenas adaptações no algoritmo do *Gradient Descent* (trocar o sinal de menos pelo sinal de mais). Outra observação importante é que muitas funções que poderíamos querer minimizar não são convexas e isso pode desembocar em situações que há mais de um ponto de mínimo (problema!!!). No entanto, aqui focamos em funções convexas e que, consequentemente, têm somente um ponto de mínimo. Os algoritmos mais simples de *Machine Learning* não sofrem com esse problema e os mais complexos, como as Redes Neurais, podem ou não sofrer com o problema - assunto para o futuro!

3.3.6 Materiais complementares

Se você achou o material muito básico até aqui, sugiro que leia o capítulo `http://www.deeplearningbook.org/contents/numerical.html`. Outros materiais de apoio:

- `https://www.youtube.com/playlist?list=PL2Wzg8U7YXS9IganqhtPS2YSgZ8dtAM3N`
- `https://www.youtube.com/playlist?list=PL2Wzg8U7YXS9HjwRCy_1kcAMHLFUoV9tu`
- `https://en.wikipedia.org/wiki/Limit_(mathematics)`
- `https://en.wikipedia.org/wiki/Derivative`
- `https://en.wikipedia.org/wiki/Partial_derivative`
- `https://en.wikipedia.org/wiki/Gradient`
- `https://en.wikipedia.org/wiki/Gradient_descent`

neuron



4. Aprendizado de Máquina

Com o avanço da tecnologia, o poder de processamento computacional está cada vez maior e, em função disso, os computadores começaram a aprender com algoritmos a automatização de extração de conhecimentos com muito mais poder e rapidez. Hoje em dia o Aprendizado de Máquina (ou *Machine Learning*) está presente em nossos celulares e computadores, usando algoritmos que aprendem interativamente a partir de dados. A partir de agora, vamos entrar mais a fundo no aprendizado de máquina e como ele pode ser utilizado. Veremos quais são os principais métodos de aprendizado mais a frente.

4.1 Tipos de Aprendizado de Máquina

Os principais tipos de aprendizado de máquina são: aprendizagem supervisionada, não supervisionada e semi-supervisionada. A seguir observamos a característica de cada uma delas. De certa forma, todos os tipos de aprendizado são baseados em experiências passadas (dados) e infere algo sobre a realidade, tentando resolver um problema. Resumidamente, os algoritmos utilizam reconhecimento de padrões e grande quantidade de dados para obter respostas precisas. Exemplos: recomendações do Netflix, Reconhecimento de face do Facebook, carro autônomo da Google.

4.2 O Aprendizado Não Supervisionado

Neste tipo de aprendizado não existe supervisão ou "professor" que fornece os resultados corretos de saída, apenas os dados de entrada. O algoritmo busca por padrões e procura saber quando a característica ocorre e quando não ocorre, ou seja, tem liberdade de classificar como quiser. O conjunto das técnicas não supervisionadas e mais utilizadas para agrupar os dados que são semelhantes se chama *clustering*, por exemplo.

4.2.1 Métodos de Aprendizado Não Supervisionado

Os principais objetivos do aprendizado não supervisionado, são encontrar estruturas em dados não classificados, padrões inesperados e classificar os dados em grupos por alguma similaridade. Nesse

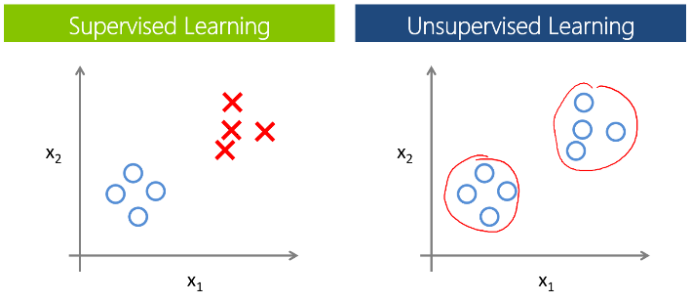


Figura 4.1: Exemplo das diferentes abordagens de aprendizado, a primeira mostra o supervisionado: onde as figuras são separadas de acordo com um atributo já conhecido; enquanto a segunda mostra o aprendizado não supervisionado, onde ocorre a classificação de acordo com características em comum. *Disponível em: https://lakshaysuri.files.wordpress.com/2017/03/sup-vs-unsup.png?w=648*

estudo explicaremos como funcionam os algoritmos de algumas técnicas de Clustering e redução de dimensionalidade, que englobam classificação atributos em grupos e a diminuição do número de variáveis usadas sem uma perda de informação relevante. Dentre esses métodos abordaremos; o Métodos de Particionamento (*Partitioning Method*), Classificação Hierárquica (*Hierarchical Clustering*) e Análise do Componente Principal (*Principal Components Analysis – PCA*).

4.2.2 Métodos de Particionamento

Esses métodos são baseados na separação dos indivíduos em subgrupos, sendo que uma parte importante desse tipo de algoritmo é o número de subgrupos k que serão formados e que deve ser informado pelo usuário. Para encontrar o melhor k, usamos métricas que descrevem a melhor estimativa dos grupos em relação aos dados. Dentre os métodos de particionamento, abordaremos o *k-means* e o PAM.

Podemos fazer o uso do algoritmo de *K means* quando possuo um conjunto de dados não rotulados e gostaria de classificá-los, para assim poder observar padrões nos dados. Posteriormente, poderia ocorrer uma análise mais aprofundada para descobrir o motivo de tal agrupamento, caso este aconteça.

Por exemplo, pensem na base de dados Íris o qual já trabalhamos. Ela é composta por quatro atributos com valores referentes às medidas da flor da orquídea e uma variável que nos diz qual seria o tipo de orquídea que contém aquelas medidas, ou seja, o tipo da planta. No aprendizado não supervisionado nós não teríamos essa última variável, a qual nos diz qual seria a espécie dessa orquídea. Logo, se caso quisermos descobrir se existe algum padrão nos dados, não podemos afirmar com muita exatidão que esses grupos descobertos existem de fato.

O primeiro passo do algoritmo *k-means* é a normalização dos dados, pois dados não normalizados atrapalham no processo de classificação. O segundo passo envolve o cálculo das distâncias entre todos os indivíduos da base de dados de acordo com suas características, usando por exemplo a distância euclidiana. Logo após, informarmos o valor de K, que é o número de grupos em que os indivíduos devem ser classificados.

O algoritmo faz uso de centroides, que são pontos centrais. Esses centroides são pontos onde os dados tomaram como referência para o agrupamento, sendo a quantidade de *centroides* = K. Os valores dos centróides são passados inicialmente de forma aleatória. Para toda observação é calculado, por meio de distância, qual seria o centróide mais próximo do indivíduo inserido, assim

a este é classificado por proximidade do centróide. Em seguida, o valor para os centroides são recalculados, sendo agora a média dos valores classificados na em cada classe associada a um centroide. Assim o algoritmo continua recursivamente até convergir, como podemos ver na figura 4.2: [1]

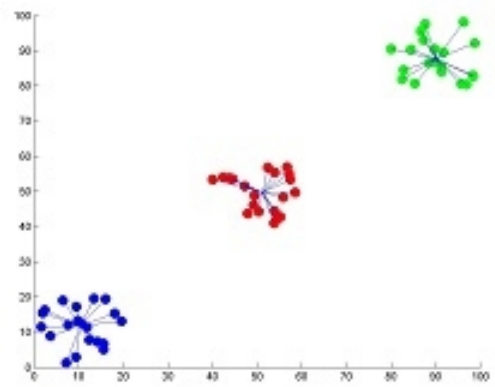


Figura 4.2: Demonstração do algoritmo de *K-means*, onde podemos ver os centróides, ao centro de cada agrupamento de dados.

Importante ressaltar que o valor de K é dado pelo usuário, portanto deve-se escolher o que melhor se aplica ao dataset usado, entretanto poderá ocorrer variações no cluster, contudo o dado vai ser classificado no grupo em que apresentar a menor distância até o centroide (Pichiliani, 2015)[2]. A seguir está a representação do pseudocódigo do *K-means* quando o o centroide inicia a partir de observações já presentes nos dados (ou conjunto D, como colocado abaixo) e não a partir de uma distribuição de probabilidade. [3]

Input: *k* (the number of clusters),
D (a set of lift ratios)
Output: a set of k clusters
Method:
Arbitrarily choose *k* objects from *D* as the initial cluster centers;
Repeat:
1. (re)assign each object to the cluster to which the object is the most similar, based on the mean value of the objects in the cluster;
2. Update the cluster means, i.e., calculate the mean value of the objects for each cluster

Until no change;

Figura 4.3: Pseudocódigo do algoritmo de k-means.

[1] https://sites.google.com/site/dataclusteringalgorithms/_/rsrc/1273871582832/k-means-clustering-algorithm/k-means.jpg

[2] https://www.devmedia.com.br/data-mining-na-pratica-algoritmo-k-means/4584

[3] Disponívelemhttps://www.researchgate.net/figure/The-pseudo-code-for-K-means-clustering-algorithm_fig2_273063437.

O algoritmo PAM ou *k-medoids*, segue o mesmo princípio do *k-means*, entretanto o PAM não é sensível a outliers, que são valores que não estão de acordo com o restante dos dados - não caracterizam bem o conjunto de atributos.

O método de particionamento pode ser utilizado para agrupar genes que são expressos em determinados períodos em uma mesma célula, por exemplo. Um vídeo muito bom sobre o método pode ser acessado em `https://www.youtube.com/watch?v=RD0nNK51Fp8`.

4.2.3 Classificação Hierárquica

A ideia principal desse tipo de classificador é construir uma estrutura hierárquica de indivíduos que possuem proximidade, o resultado desse algoritmo é um dendrograma, que é uma árvore onde os dados são divididos.

Esse algoritmo de clusterização utilizará as distâncias entre os indivíduos de acordo com atributos pré-definidos. O algoritmo primeiramente forma pares de dois indivíduos mais próximos e considera esse par como sendo um único cluster. Depois de haver o pareamento dois a dois (sendo que alguns indivíduos podem não ter sido pareado com nenhum outro - consideraremos nesse caso que o indivíduo é um cluster), computa-se o centroide de cada cluster formado e as distâncias entre os centroides de todos os clusters são recalculadas. Após o calculo das distâncias os clusters são pareados dois a dois, repetindo o processo anterior. Os novos centroides são calcuados como uma média dos atributos de todos os indivíduos dentro do cluster.

A regra de parada deve ser definida pelo usuário. O processo é repetido até que se forme um número pré-definido de clusters ou até que os novos clusters formados tenham características que são consideradas ruins[4]. Podemos observar o exemplo de dendrograma [5] abaixo:

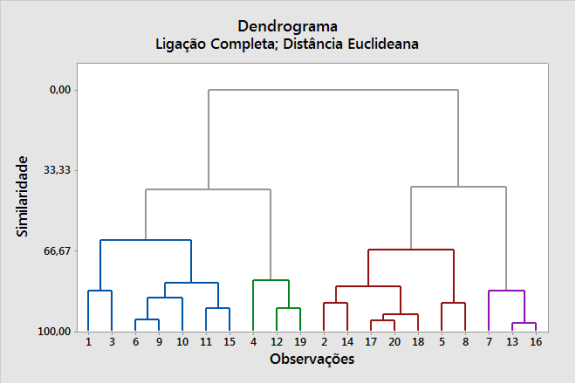


Figura 4.4: Resultado da Classificação Hierárquica de um conjunto de observações.

Podemos utilizar esse algoritmo para classificar genomas de pacientes por exemplo. Onde os nós folhas representam os pacientes, e conseguimos predizer o quanto cada dado está relacionado formando assim o Dendrograma.

Podemos ver o pseudocódigo do algoritmo abaixo [6], se trata de um algoritmo de simples

[4] Ver `https://www.youtube.com/watch?v=rg2cjfMsCk4`

[5] `https://support.minitab.com/pt-br/minitab/18/cluster_obs_dendrogram_with_final_partition_glove_testers.png`

[6] `Disponívelemhttps://www.researchgate.net/profile/Halina_Kwasnicka/publication/202144697/figure/fig1/AS:340534259404805@1458201102592/The-hierarchical-clustering-algorithm-in-pseudocode.png`

compreensão:

1. Turn each input element into a singleton, i.e, into a cluster of a single element.
2. For each pair of clusters c1, c2 calculate their distance *d*(c1, c2).
3. Merge the pair of clusters that take the smallest distance.
4. Continue the step 2, until the termination criterion is satisfied.
5. The termination criterion most commonly used is a threshold of the distance value

Figura 4.5: Pseudocódigo do algoritmo de Classificação Hierárquica.

Esse algoritmo apresenta como vantagem em relação ao *k-means* e ao PAM, a falta de necessidade de especificar o número de *clusters*, entretanto pode ocorrer alterações dependendo do cálculo escolhido para a distância. A classificação hierárquica pode ser utilizado no alinhamento global de sequências, demonstrando quais têm maior relação dentre todas.

4.2.4 *Principal Components Analysis* - PCA

O método PCA é utilizado quando há a necessidade de se reduzir a quantidade de atributos de dados numéricos enquanto tenta-se manter a estrutura dos dados. Podemos usar o PCA quando trabalhamos com análise de genes diferencialmente expressos, onde a quantidade de dados é muito grande e não queremos que ocorra a perda de informações, por exemplo. Podemos observar a figura 4.6, para uma visão mais geométrica do modelo [7]:

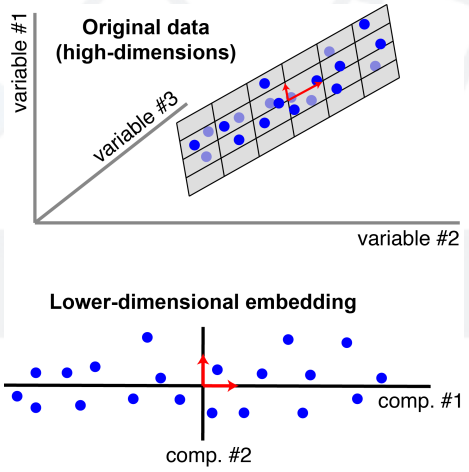


Figura 4.6: Exemplo de utilização do PCA para minimizar o número de componentes.

Intuitivamente, o método do PCA encontra as direções que os dados têm maior variação e "reconstrói"as variáveis nessas direções. Na imagem acima, temos 3 variáveis, no entanto, percebe-se que os dados têm grande parte de suas variações em apenas duas direções. Assim sendo, podemos reconstruir as variáveis nas direções que explicam grande parte das variações nos dados. As duas novas variáveis são independentes entre si e são calculadas a parte de combinações das três originais

[7] http://alexhwilliams.info/itsneuronalblog/2016/03/27/pca/

de modo a maximizar a variância desses dados. É importante dizer que antes de se realizar o PCA, é necessário reescalar os dados para que todas as variáveis tenham a mesma escala. O número de componentes (ou variáveis novas) utilizados deve ser escolhido pelo usuário de acordo com o quanto da variação pode ser explicada pelos novos componentes em relação às variáveis originais. Mais detalhes podem ser acessados em https://pt.wikipedia.org/wiki/Análise_de_componentes_principais.

4.3 Materiais complementares

Se você achou o material muito básico até aqui, sugiro que leia o capítulo http://www.ee.columbia.edu/~vittorio/UnsupervisedLearning.pdf. Outros materiais de apoio:

- https://medium.com/machine-learning-for-humans/unsupervised-learning-f45587588294
- https://www.youtube.com/playlist?list=PL2Wzg8U7YXS9HjwRCy_1kcAMHLFUoV9tu
- http://stanford.edu/~cpiech/cs221/handouts/kmeans.html
- https://www.cs.utah.edu/~piyush/teaching/4-10-print.pdf
- http://aquilesburlamaqui.wdfiles.com/local--files/logica-aplicada-a-computacao/texto_fuzzy.pdf
- https://www.analyticsvidhya.com/blog/2016/03/practical-guide-principal-component-analysi
- https://www.toptal.com/machine-learning/clustering-algorithms
- http://www.teses.usp.br/teses/disponiveis/55/55134/tde-19082004-092311/.

neuron